ÇANKAYA UNIVERSITY
COMPUTER ENGINEERING
DEPARTMENT

CENG 408

System Architecture and Methodology

TARAS: Agricultural Analysis System

ARDA KAHRAMAN - 202111002
HATİCE ELİF ARABACI - 202211008
ÖYKÜ EYLÜL BOZDAĞ - 202311410
MUSTAFA BERK KESKİN - 202211042

TABLE OF CONTENTS

1. Introduction

1.1. Purpose

This document defines the system architecture of the TARAS precision agriculture platform by presenting its core components, logical layers, and interactions. The architectural decisions are derived from the functional and non-functional requirements defined in the SRS.

In addition to the architectural design, the document explains the methodological approach for data acquisition, preprocessing, and irrigation decision logic, showing how agronomic principles and sensor-driven analytics are translated into system behavior. It also describes auxiliary AI-assisted advisory components that support explainability and user understanding without performing core decision-making.

Overall, this document serves as an architectural and methodological reference for implementation, integration, and future development, ensuring a consistent, maintainable realization of the TARAS platform.

1.2. Definitions and Acronyms

Term	Definition
Accuracy	The ratio of correctly classified instances (both positive and negative) to the total number of instances. It represents the overall correctness of the model's predictions.
Precision	The proportion of correctly predicted positive instances among all instances predicted as positive. It indicates how reliable the positive predictions of the model are.
Recall	The proportion of correctly predicted positive instances among all actual positive instances. It measures the model's ability to identify all relevant positive cases.
F1-Score	The harmonic mean of precision and recall. It provides a balanced measure that considers both false positives and false negatives, especially useful when class distributions are imbalanced.
Agronomy	The branch of agricultural science that focuses on crop production and soil management, examining the interactions between crops, soil, climate, and agricultural practices in order to optimize yield and resource efficiency.

JWT(JSON Web Token)	JWT is a compact and secure token format used for authentication and authorization, allowing the system to verify user identity and access rights between the client and backend services.
ML	ML refers to computational techniques that enable systems to learn patterns from data and make predictions or decisions without being explicitly programmed, and is used in the TARAS system primarily for plant disease detection and auxiliary analysis tasks.
CV Models	CV models are computational models designed to analyze and interpret visual data such as images, enabling tasks like object recognition, classification, and pattern detection. In the TARAS system, they are used for plant leaf disease detection.
Cloud Services	Cloud services refer to on-demand computing resources such as storage, databases, and backend services provided over the internet, used in the TARAS system to support data storage, processing, and system scalability.
Client	The client is the user-facing component of a client–server architecture that initiates requests to the server and presents system functionality and data to the user. In the TARAS system, the mobile application acts as the client.
Server	The server is the backend component of a client–server architecture that receives requests from clients, processes them, manages data storage and system logic, and returns responses. In the TARAS system, the server hosts backend services, databases, and analysis modules.
Storage Bucket	A storage bucket is a logical container in cloud storage systems used to store and organize unstructured data such as images and files. In the TARAS system, storage buckets are used to store uploaded plant images and related assets.
Classification	Classification is a ML task in which input data are assigned to one of several predefined categories or classes. In the TARAS system, classification is used to identify plant diseases from leaf images.
Processing Pipeline	The process of collecting raw data from heterogeneous sources and transforming it into a consistent, reliable, and analysis-ready format. This includes acquiring environmental sensor data, historical records, metadata, and image data, followed by preprocessing steps such as timestamp alignment, unit normalization, quality control, outlier detection, and missing data handling to ensure data integrity before model-based analysis.

Deep Learning	A subfield of ML that utilizes neural networks with multiple layers to automatically learn hierarchical feature representations from data. It is particularly effective for complex tasks such as image recognition, natural language processing, and speech analysis.
CNN	A deep learning architecture designed to automatically extract features from image data using convolutional layers. It is widely used in tasks such as image classification and object recognition.
Overfitting	Overfitting refers to a modeling phenomenon where a ML model learns training data too closely, resulting in high training performance but poor generalization to unseen data.
Pre-trained weights	Model parameters obtained by training a neural network on a large-scale dataset prior to being applied to a target task. They are used to accelerate training and improve generalization performance.
Finetune	The process of retraining some or all layers of a pre-trained model on a task-specific dataset using a low learning rate, allowing the model to adapt to the new problem domain.
TensorFlow	An open-source ML framework developed by Google for building, training, and deploying ML and deep learning models.
Keras	A high-level deep learning API that runs on top of TensorFlow, enabling fast and efficient development of neural network models with a user-friendly and modular structure.
WCAG (Web Content Accessibility Guidelines)	WCAG is a set of international guidelines that define standards for making digital interfaces accessible to users with diverse abilities, focusing on principles such as perceivability, operability, understandability, and robustness.
Cognitive Load	Cognitive load refers to the amount of mental effort required to process information and interact with a system. In the TARAS interface design, reducing cognitive load aims to make the application easier to understand and use, especially for users with limited technical experience.
Carbon Footprint	The total amount of greenhouse gas emissions (expressed as CO_2-equivalent) produced directly and indirectly by agricultural activities.
CO_2 equivalent (CO_2e)	A standardized unit used to express the impact of different greenhouse gases in terms of the amount of CO_2 that would produce the same warming effect.

Emission Factor	A coefficient that converts activity data (such as fuel consumption or electricity use) into greenhouse gas emissions.
Greenhouse Gas (GHG)	Gases that trap heat in the atmosphere, including CO_2, CH_4 (methane), and N_2O (nitrous oxide), contributing to global warming.
Activity Data	Quantitative data representing human activities that generate emissions, such as fuel usage, electricity consumption, or fertilizer application.
Sustainability	The practice of managing resources in a way that meets current needs without compromising future environmental, economic, and social systems.
Environmental Impact	The effect of agricultural activities on the environment, including emissions, water usage, and resource consumption.
Normalized Emissions	Emission values adjusted relative to a reference unit, such as per hectare or per unit of production, to enable comparison.
Tier 1 (IPCC Methodology)	A simplified estimation method defined by IPCC that uses default emission factors and basic activity data for calculating greenhouse gas emissions.
IPCC	Intergovernmental Panel on Climate Change, an international body providing guidelines for greenhouse gas estimation.
FAO	Food and Agriculture Organization of the United Nations, providing datasets and methodologies for agriculture-related analysis.

2. System Overview

The TARAS (Tarım Analizi Sistemi) platform is a precision agriculture decision-support system designed to assist farmers in irrigation planning, crop health monitoring and sustainability assessment. The project initially explored a wide range of smart agriculture features, which were evaluated based on impact, feasibility, usability, cost and sustainability. Based on literature analysis and comparisons with existing solutions, the system scope was refined to focus on the most effective and implementable components. The final design centers on integrated field sensing, AI-driven analysis, digital twin visualization and a mobile application interface, ensuring a practical, scalable and user-oriented solution.

3. System Architecture

3.1. Architectural Design

The architectural design of the TARAS follows a hybrid architecture that combines Layered Architecture and Client–Server Architecture principles. This approach supports modular development, scalability and clear separation of responsibilities across system components.

3.1.1. Layered Architecture

3.1.1.1. Presentation Layer

Provides the mobile application interface through which farmers interact with the system. This layer presents sensor readings, irrigation recommendations, disease detection results, digital twin visualizations, sustainability metrics such as carbon footprint, and AI-assisted advisory explanations in an accessible and user-friendly manner. The advisory interface translates analytical outputs into clear, context-aware guidance, supporting user understanding without replacing farmer decision-making.

3.1.1.2. Application & Analytics Layer

Implements the core system logic, including data processing, irrigation prediction, disease analysis, decision-support workflows and user interaction management. ML and computer vision models operate within this layer to transform raw data into actionable insights.

3.1.1.3. Data & Cloud Services Layer

Handles data storage, retrieval and synchronization between field devices and the mobile application. This layer manages time-series sensor data, historical records, model outputs and user inputs using a centralized cloud-based backend.

3.1.1.4. Device & Sensing Layer

Includes IoT sensor nodes and gateways deployed in the field, responsible for collecting soil moisture, temperature and environmental data. This layer abstracts hardware-level interactions and ensures reliable data acquisition.

3.1.1.5. Communication Layer

Manages data transmission between sensor nodes, gateway devices and cloud services. Sensor nodes communicate with the

field gateway over ESP-NOW, a low-power connectionless 2.4 GHz Wi-Fi protocol, while the gateway relays buffered readings to the cloud over Wi-Fi using HTTP and a Socket.IO real-time channel. The layer ensures reliable, low-power and fault-tolerant data flow, with on-device buffering on the gateway for resilience during connectivity loss.

3.1.2. Client–Server Architecture

The TARAS mobile app operates as a client that interacts with the cloud-based backend services. Field sensor data, user inputs, and system outputs, such as irrigation recommendations, disease analysis results and sustainability metrics are exchanged through request–response communication mechanisms. This separation between client and server components enables loose coupling, supports scalability and simplifies maintenance and future extensions.

3.2. Decomposition Description

3.2.1. Field Sensing & Data Acquisition Subsystem

- Soil Moisture Sensing Module
- Environmental Data Collection (Temperature, Humidity)
- Sensor Calibration & Validation Module
- Gateway Communication Module

3.2.2. Analytics & Decision Support Subsystem

- Data Storage Module
- Irrigation Recommendation Module
- Disease Detection Module
- Sustainability & Carbon Footprint Calculation Module
- Data Processing & Model Management Module
- LLM advisory & Explainability Module

3.2.3. Visualization & User Interaction Subsystem

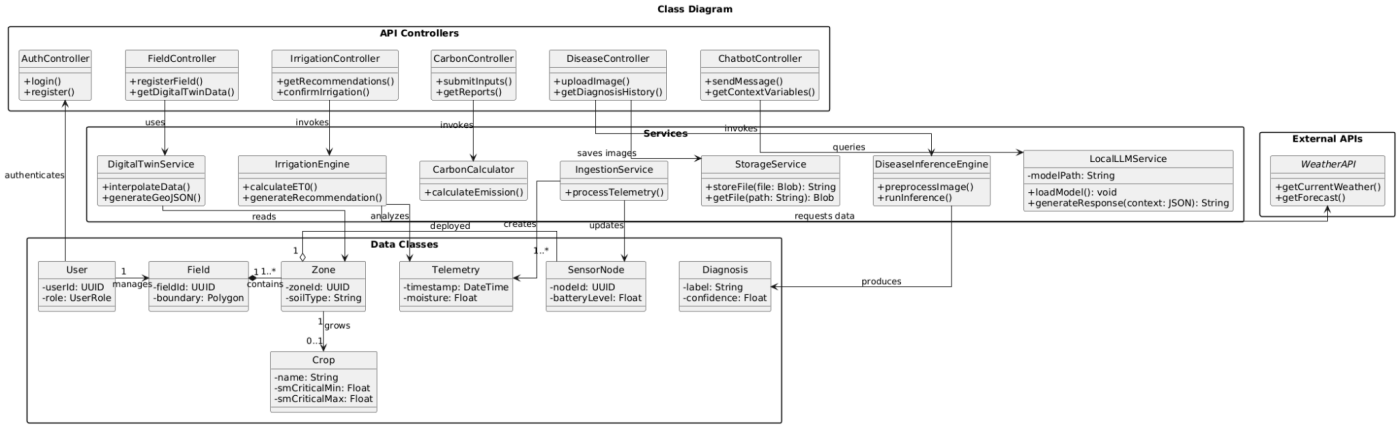
- Mobile Application Interface
- Digital Twin Visualization Module
- Recommendation & Alert Presentation
- User Input & Configuration Management

Each TARAS module communicates through clearly defined interfaces, hiding internal logic while exposing essential operations such as retrieving sensor data, generating irrigation recommendations,

performing disease analysis, and updating visualization data. This supports abstraction, modular refinement, and reusability in line with software engineering principles.

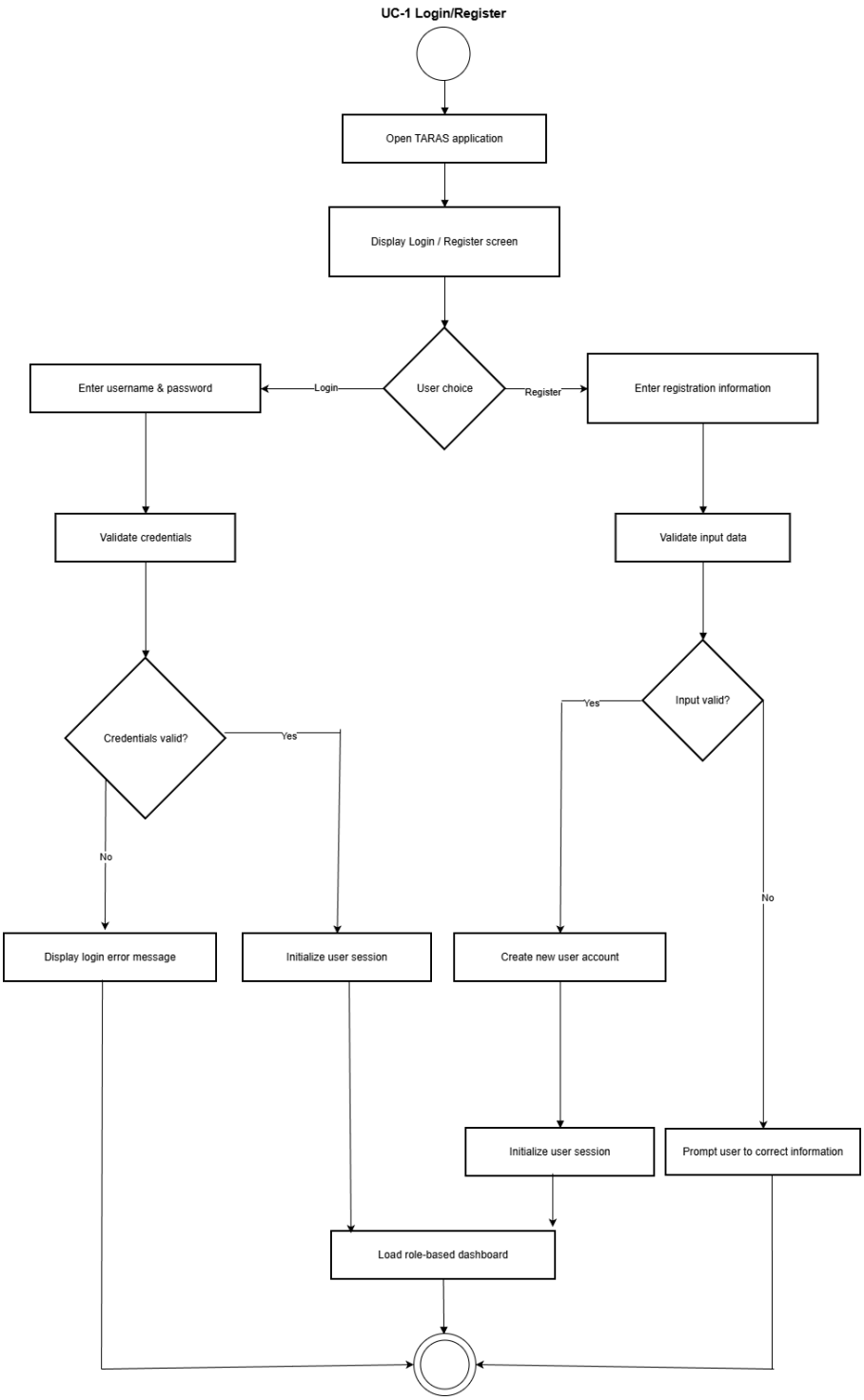
The modular structure also simplifies independent development and testing, while allowing future extensions such as new sensor types, improved prediction models, or expanded sustainability metrics without affecting the overall architecture.

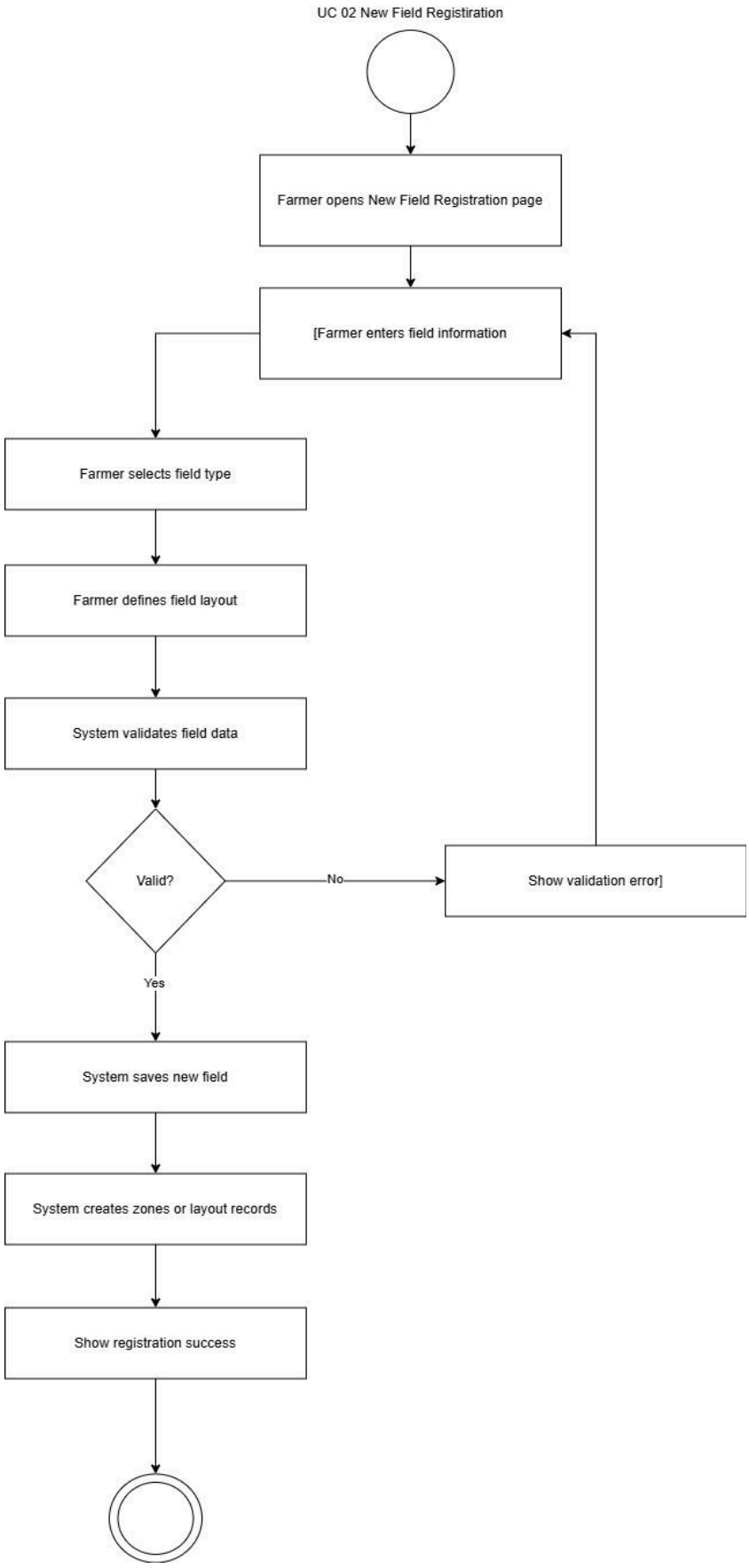
3.2.4. Class Diagram

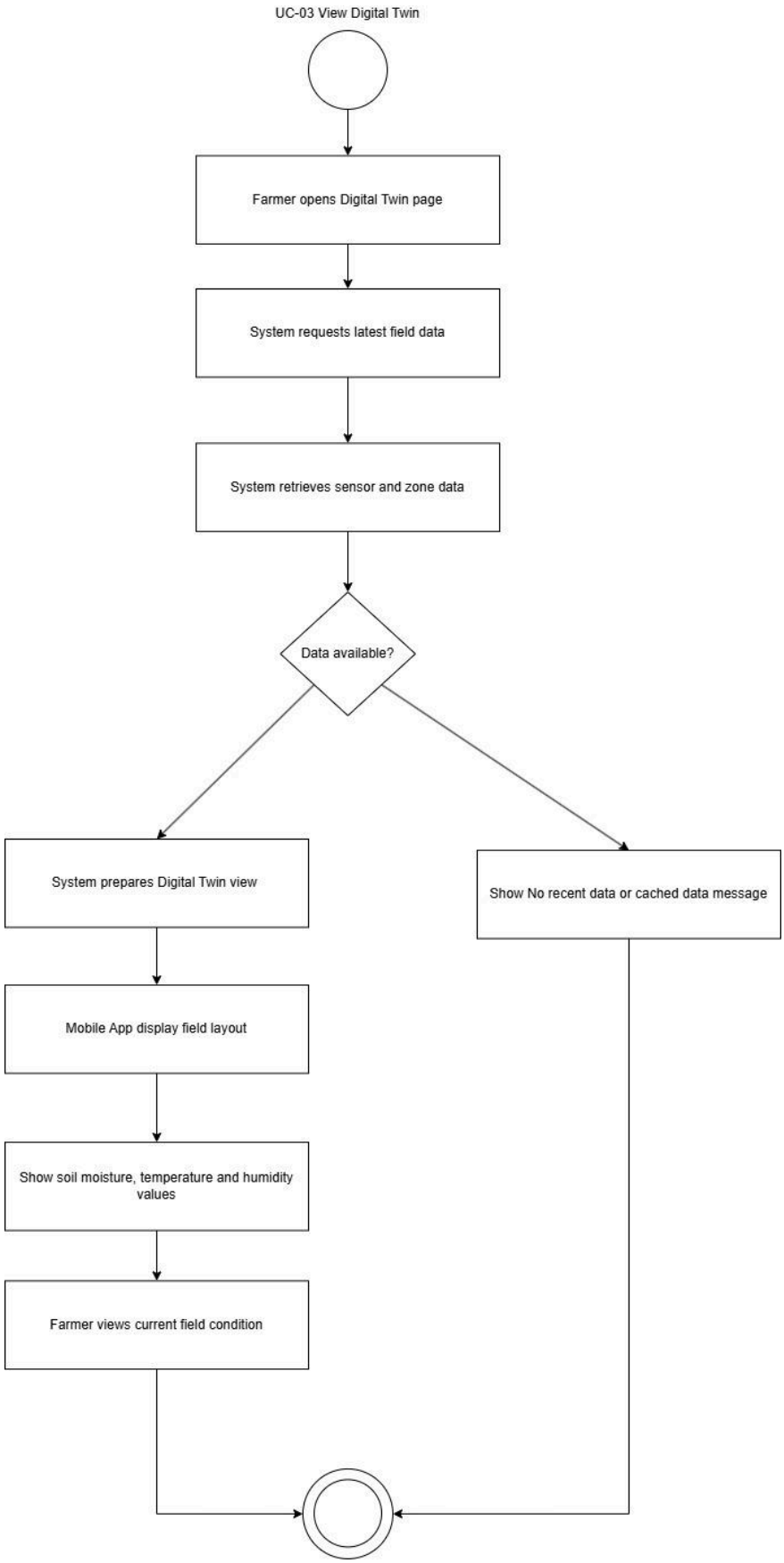


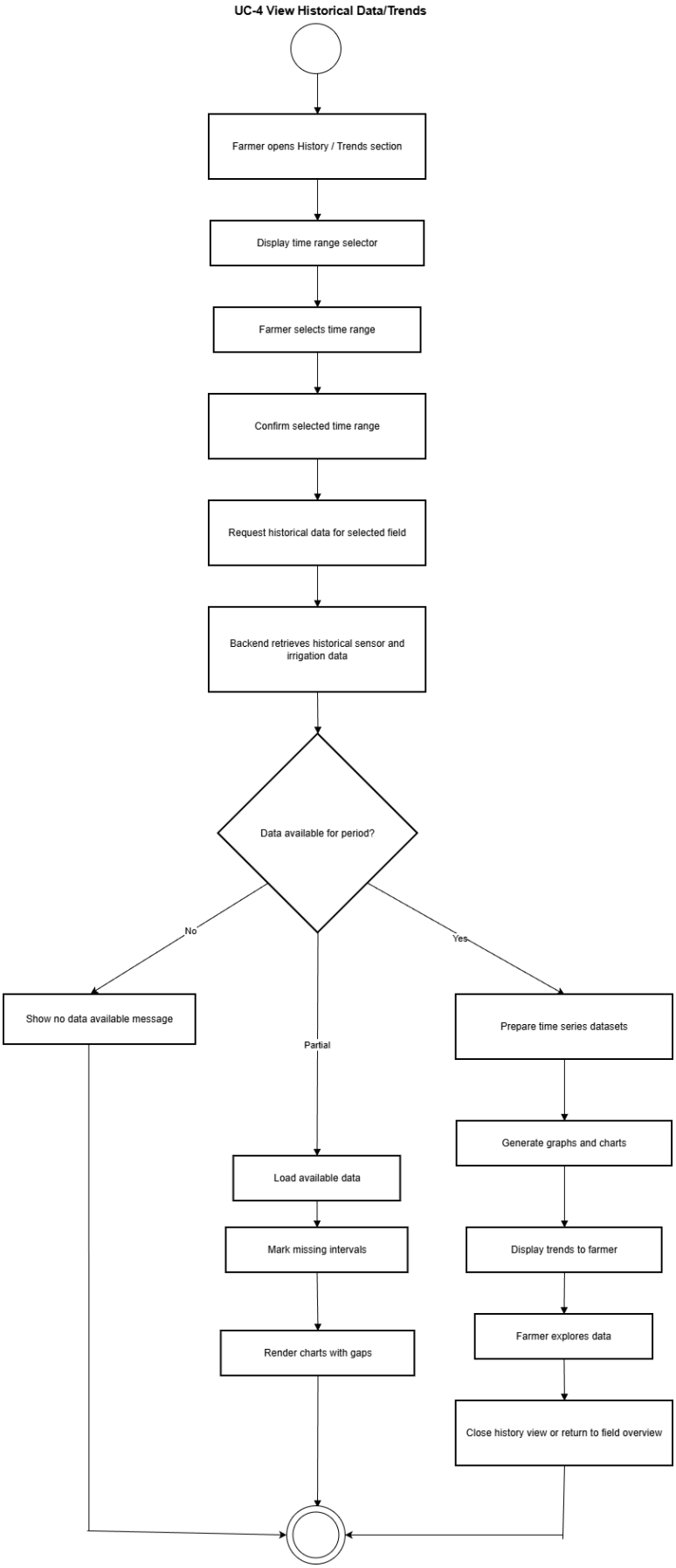
3.3. System Modeling

3.3.1. Activity Diagrams

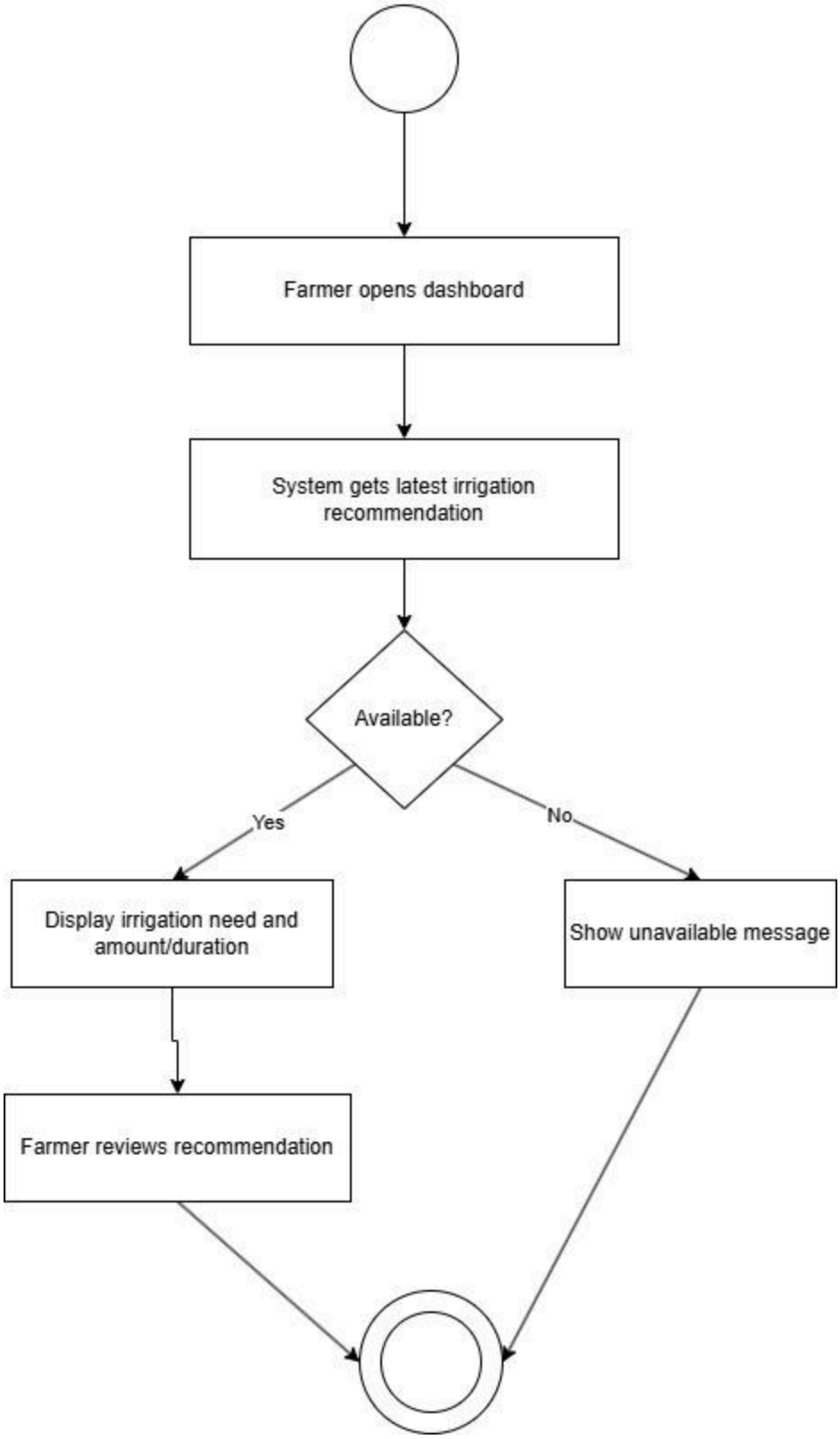


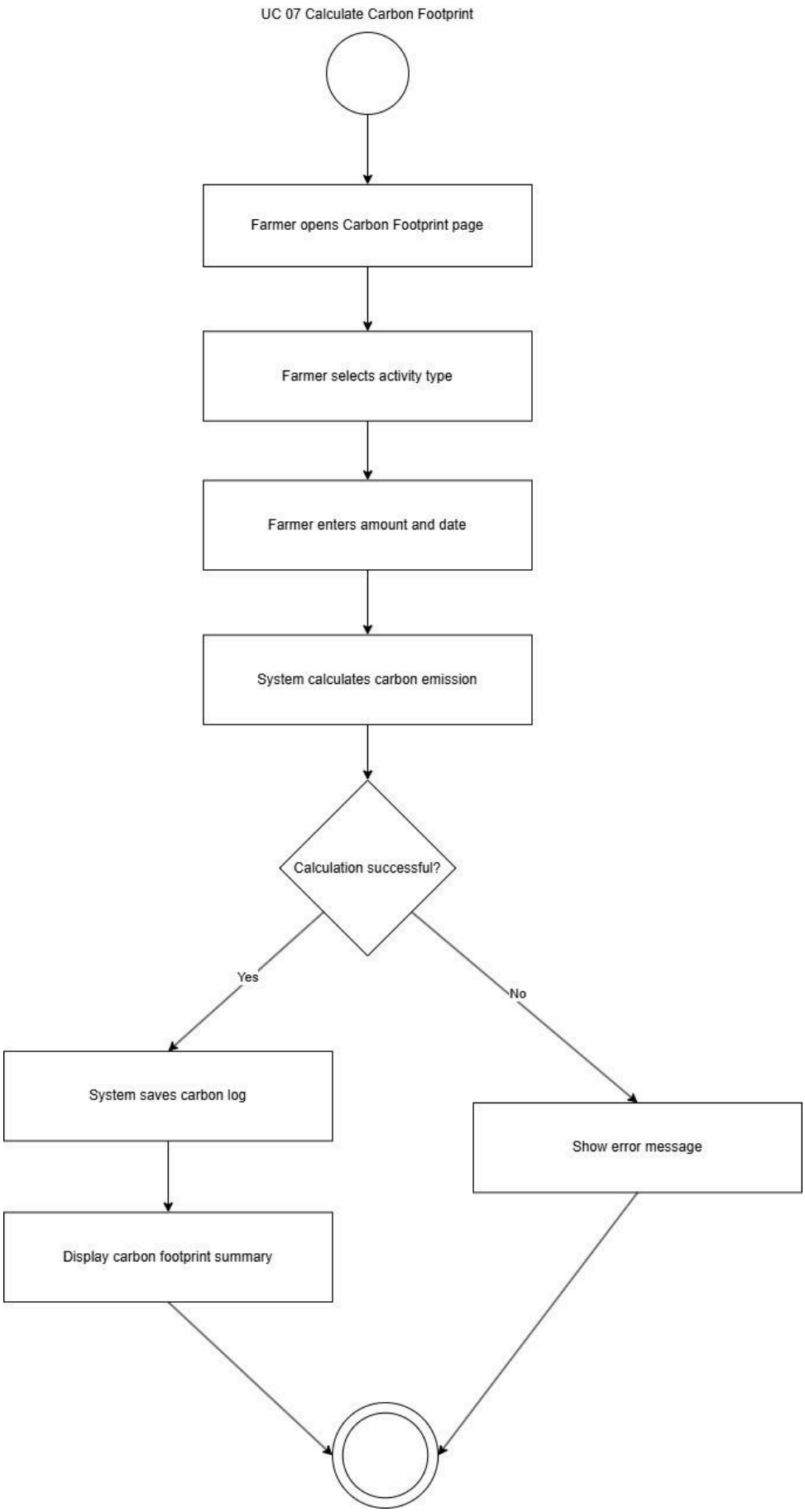

UC 02 New Field Registiration
Farmer opens New Field Registration page
[Farmer enters field information
Farmer selects field type
Farmer defines field layout
System validates field data
Valid?
No
Show validation error]
Yes
System saves new field
System creates zones or layout records
Show registration success

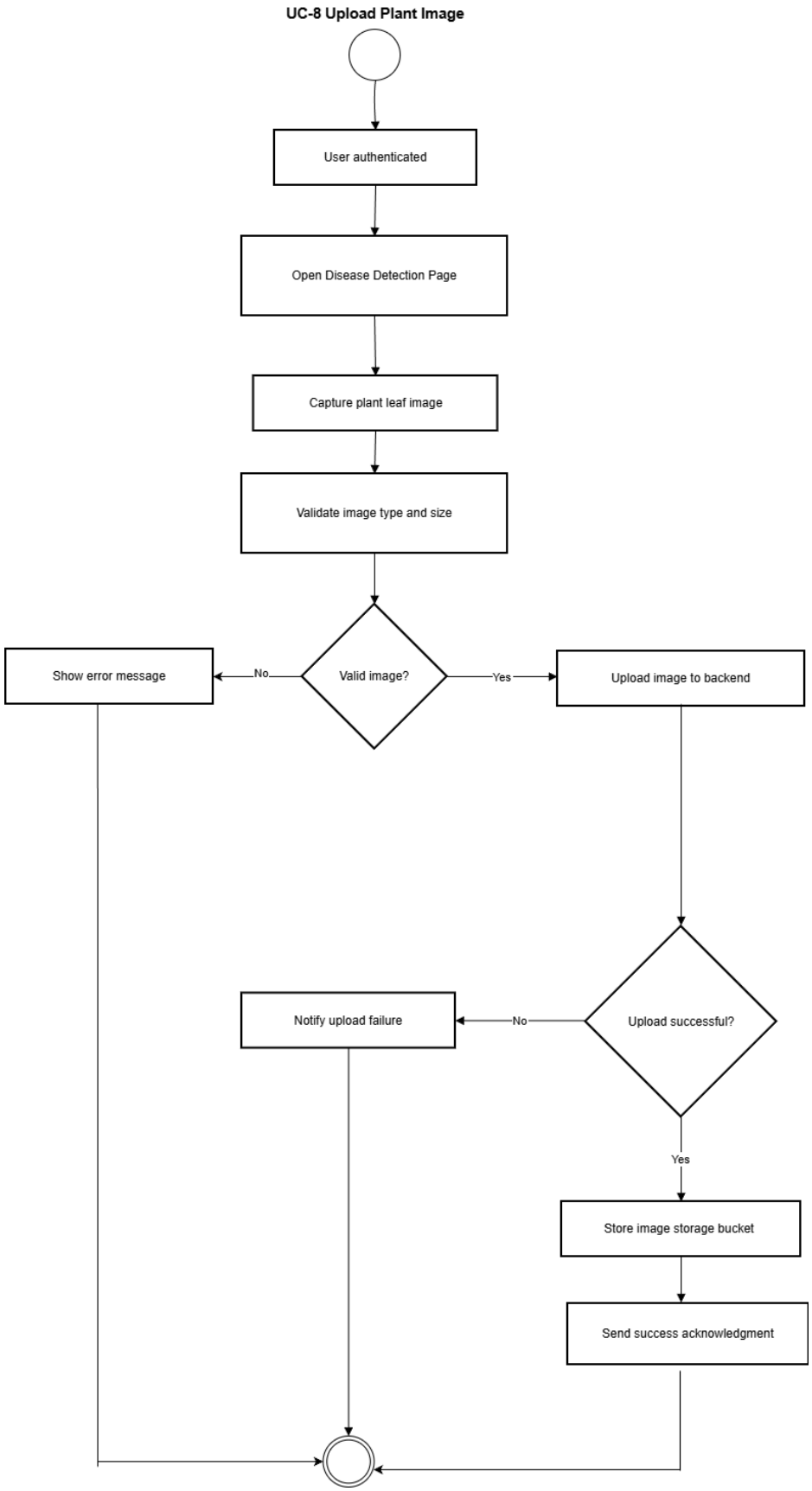

UC-03 View Digital Twin
Farmer opens Digital Twin page
System requests latest field data
System retrieves sensor and zone data
Data available?
System prepares Digital Twin view
Show No recent data or cached data message
Mobile App display field layout
Show soil moisture, temperature and humidity values
Farmer views current field condition

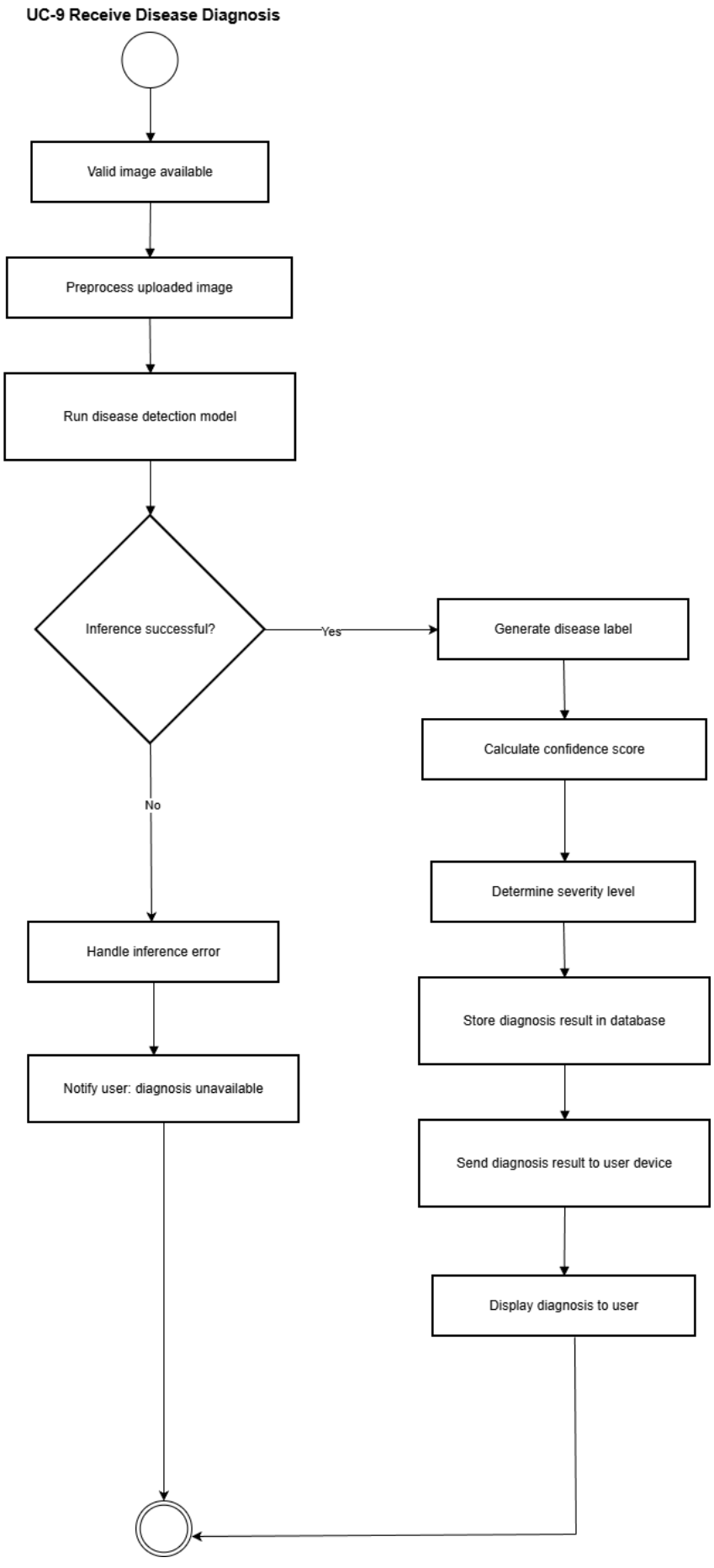

UC-4 View Historical Data/Trends
Farmer opens History / Trends section
Display time range selector
Farmer selects time range
Confirm selected time range
Request historical data for selected field
Backend retrieves historical sensor and irrigation data
Data available for period?
No
Partial
Yes
Show no data available message
Load available data
Mark missing intervals
Render charts with gaps
Prepare time series datasets
Generate graphs and charts
Display trends to farmer
Farmer explores data
Close history view or return to field overview

UC 05 Receive Irigation Suggestion

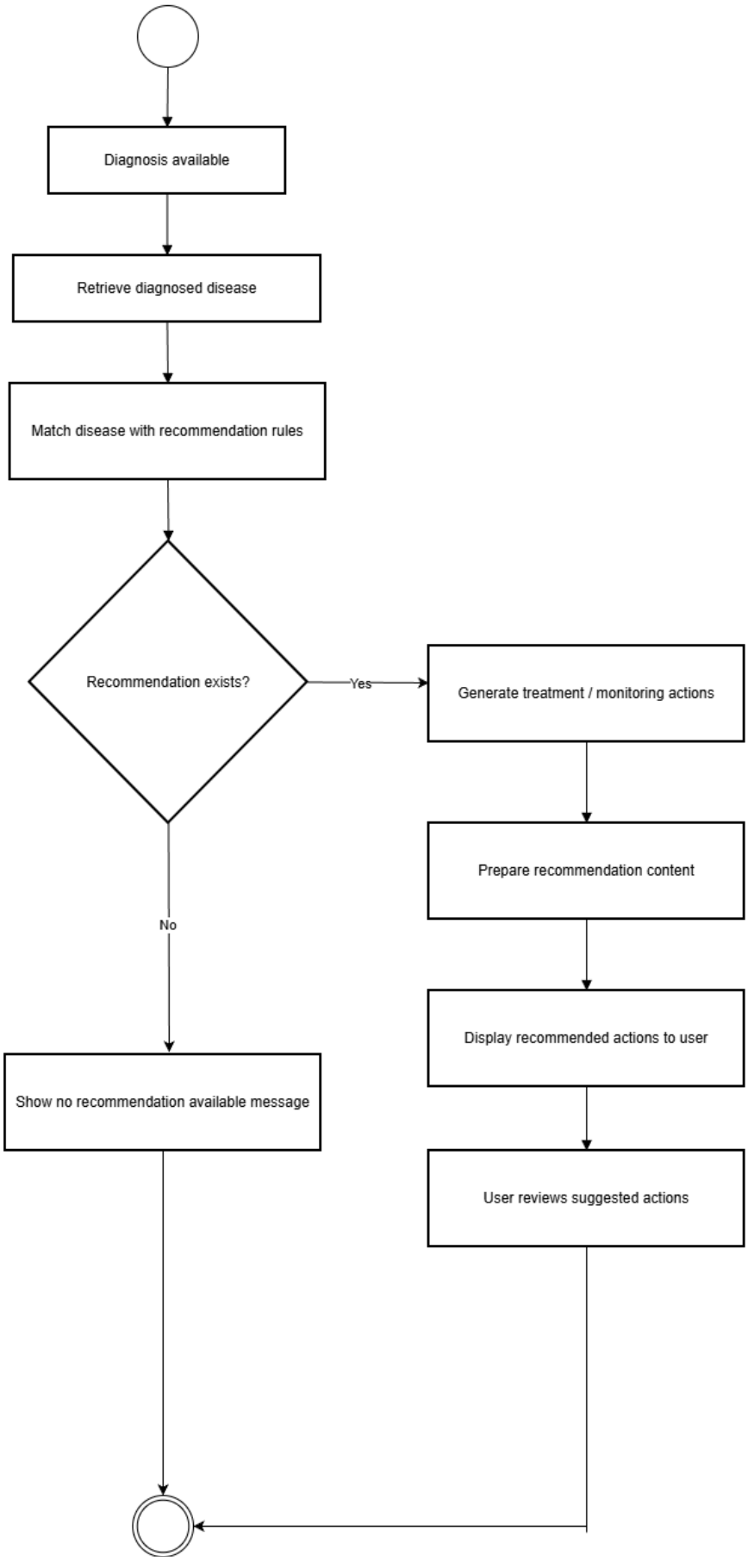


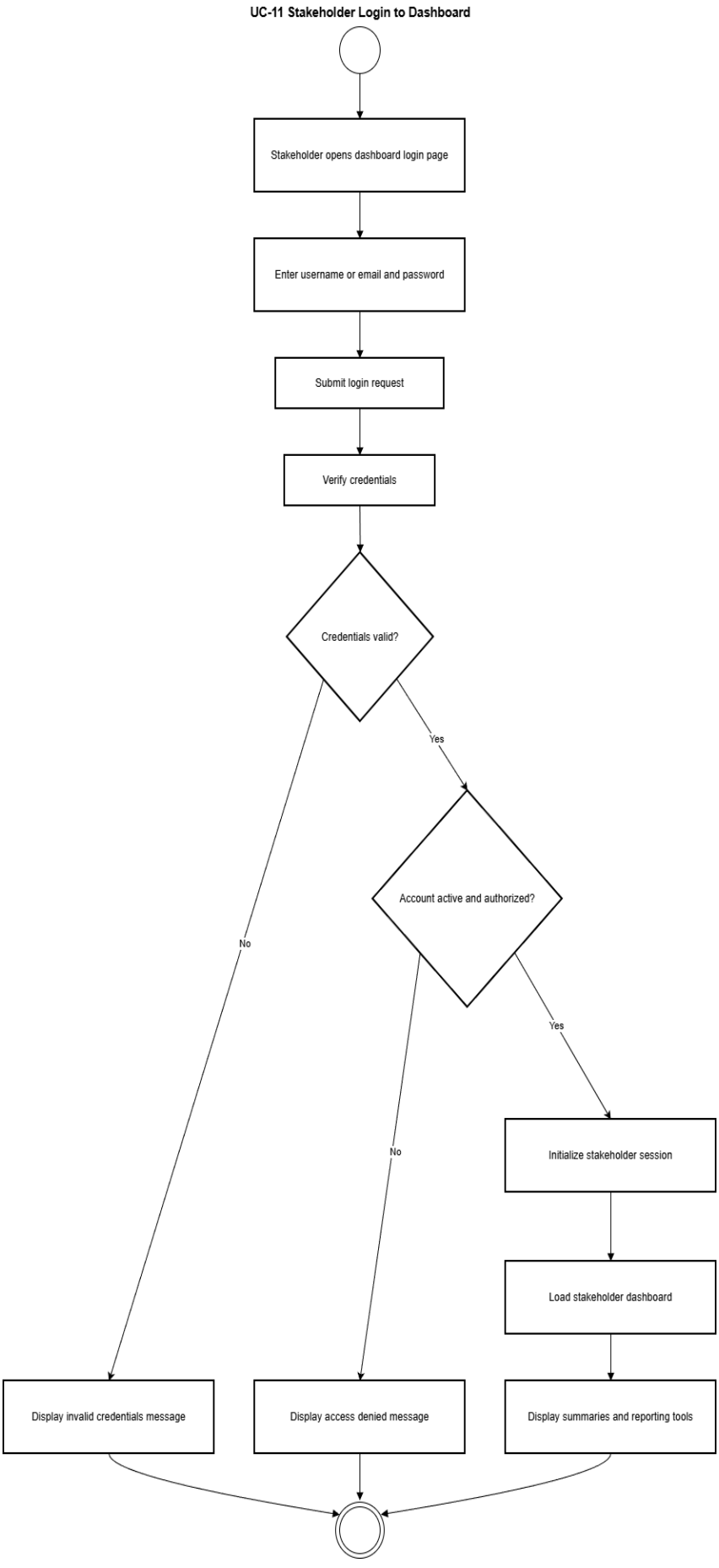

UC 07 Calculate Carbon Footprint
Farmer opens Carbon Footprint page
Farmer selects activity type
Farmer enters amount and date
System calculates carbon emission
Calculation successful?
Yes
No
System saves carbon log
Display carbon footprint summary
Show error message

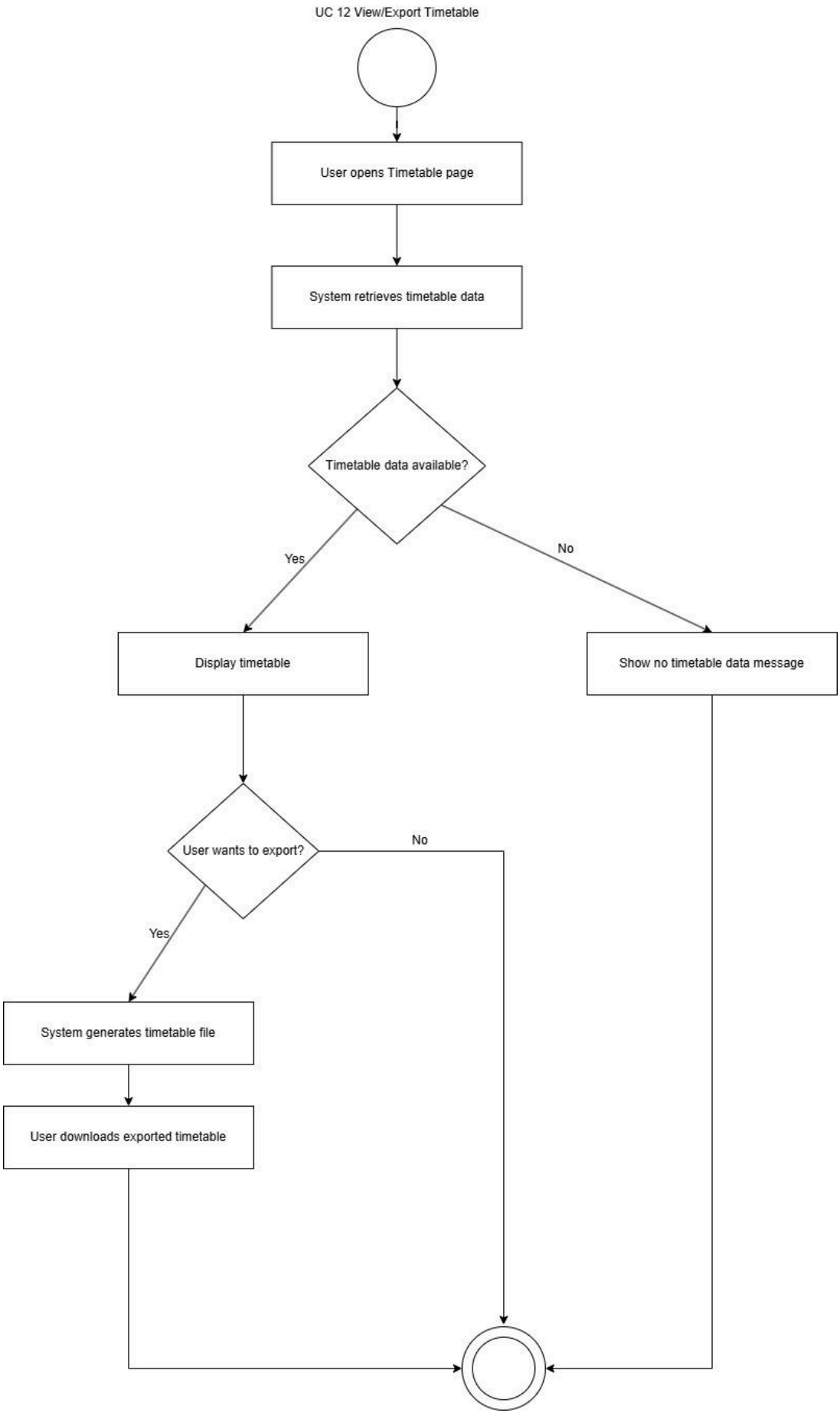

UC-8 Upload Plant Image
User authenticated
Open Disease Detection Page
Capture plant leaf image
Validate image type and size
Valid image?
No
Show error message
Yes
Upload image to backend
Upload successful?
No
Notify upload failure
Yes
Store image storage bucket
Send success acknowledgment

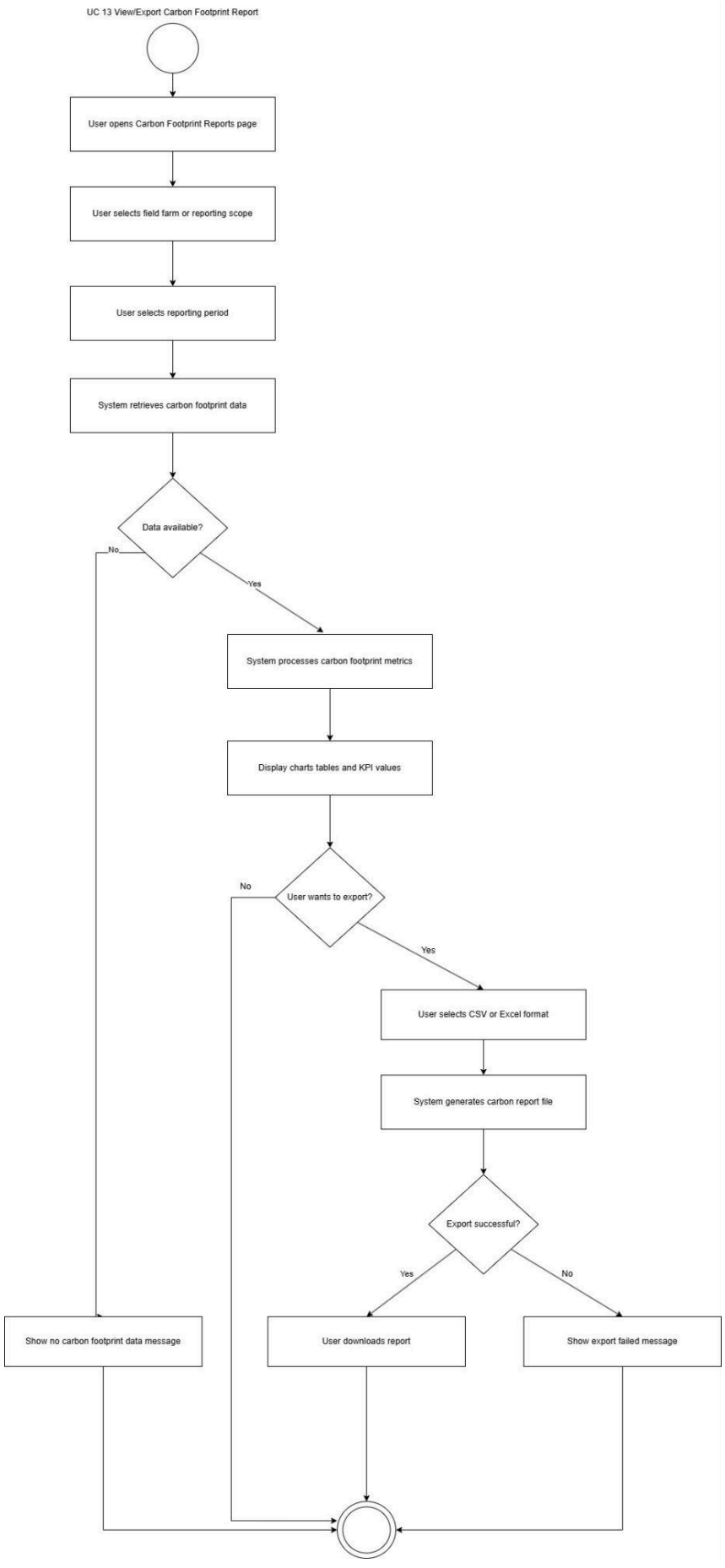

UC-9 Receive Disease Diagnosis
Valid image available
Preprocess uploaded image
Run disease detection model
Inference successful?
Yes
No
Generate disease label
Calculate confidence score
Determine severity level
Store diagnosis result in database
Send diagnosis result to user device
Display diagnosis to user
Handle inference error
Notify user: diagnosis unavailable

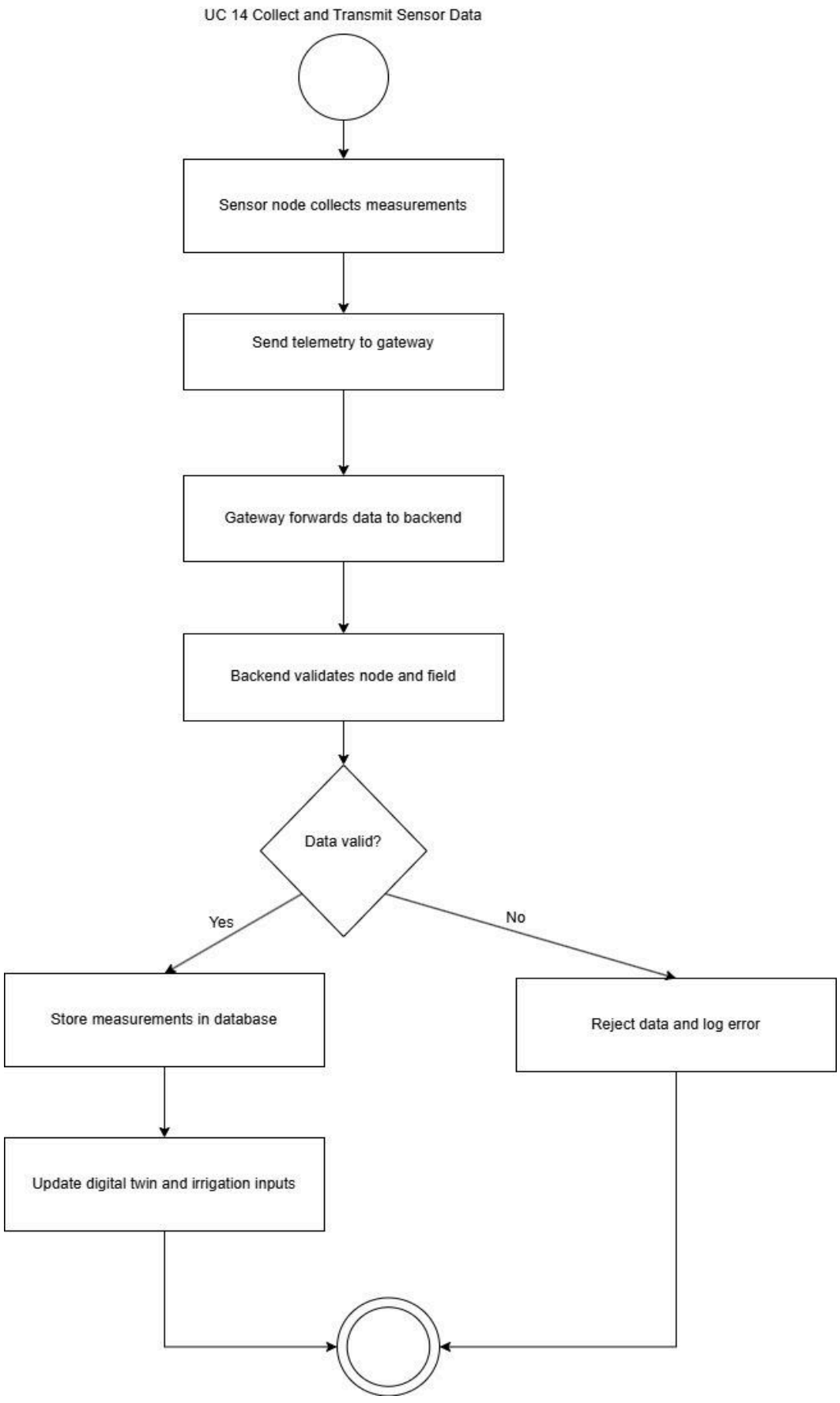
UC-10 View Recommended Actions



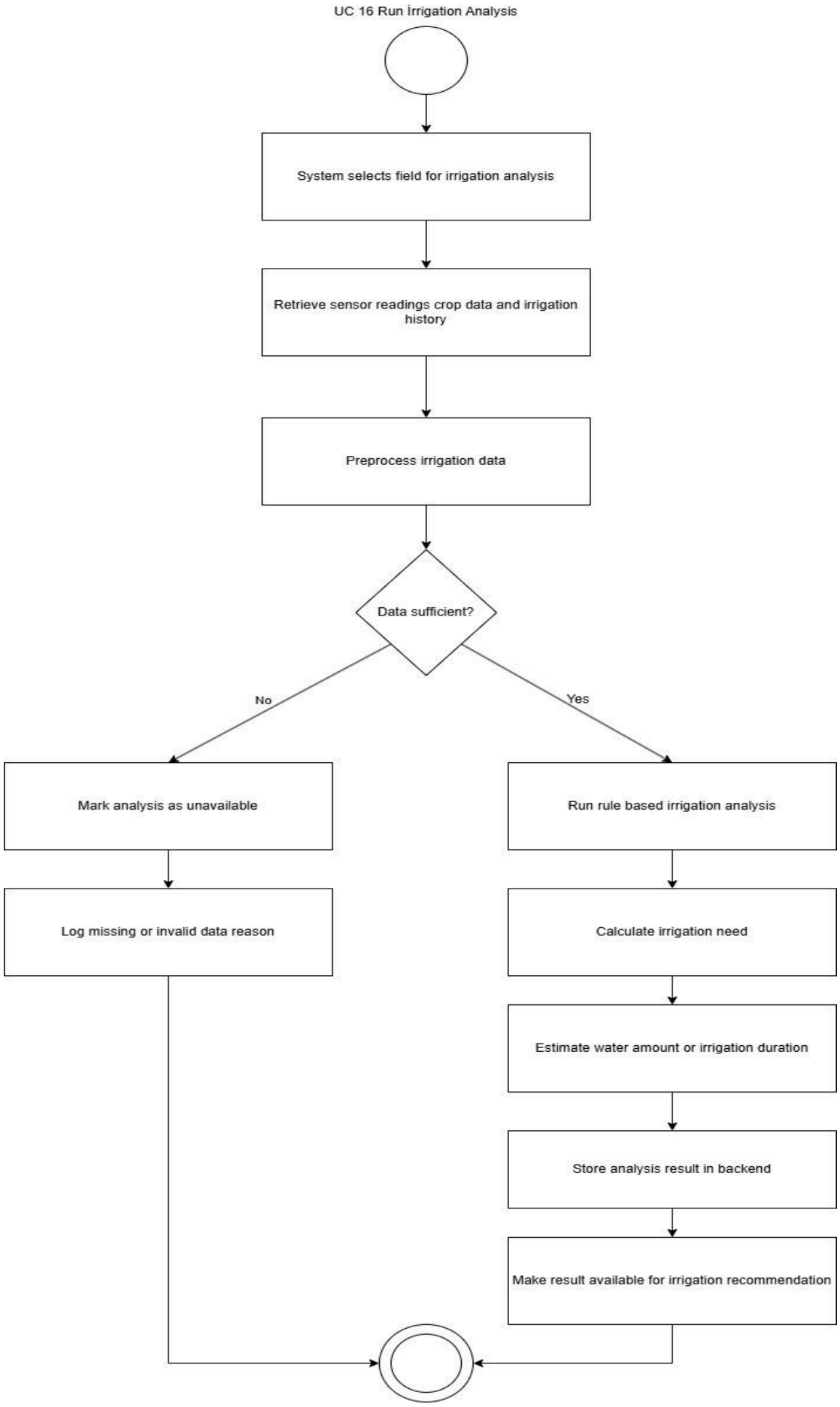

UC-11 Stakeholder Login to Dashboard
Stakeholder opens dashboard login page
Enter username or email and password
Submit login request
Verify credentials
Credentials valid?
Yes
No
Account active and authorized?
Yes
No
Initialize stakeholder session
Load stakeholder dashboard
Display summaries and reporting tools
Display invalid credentials message
Display access denied message

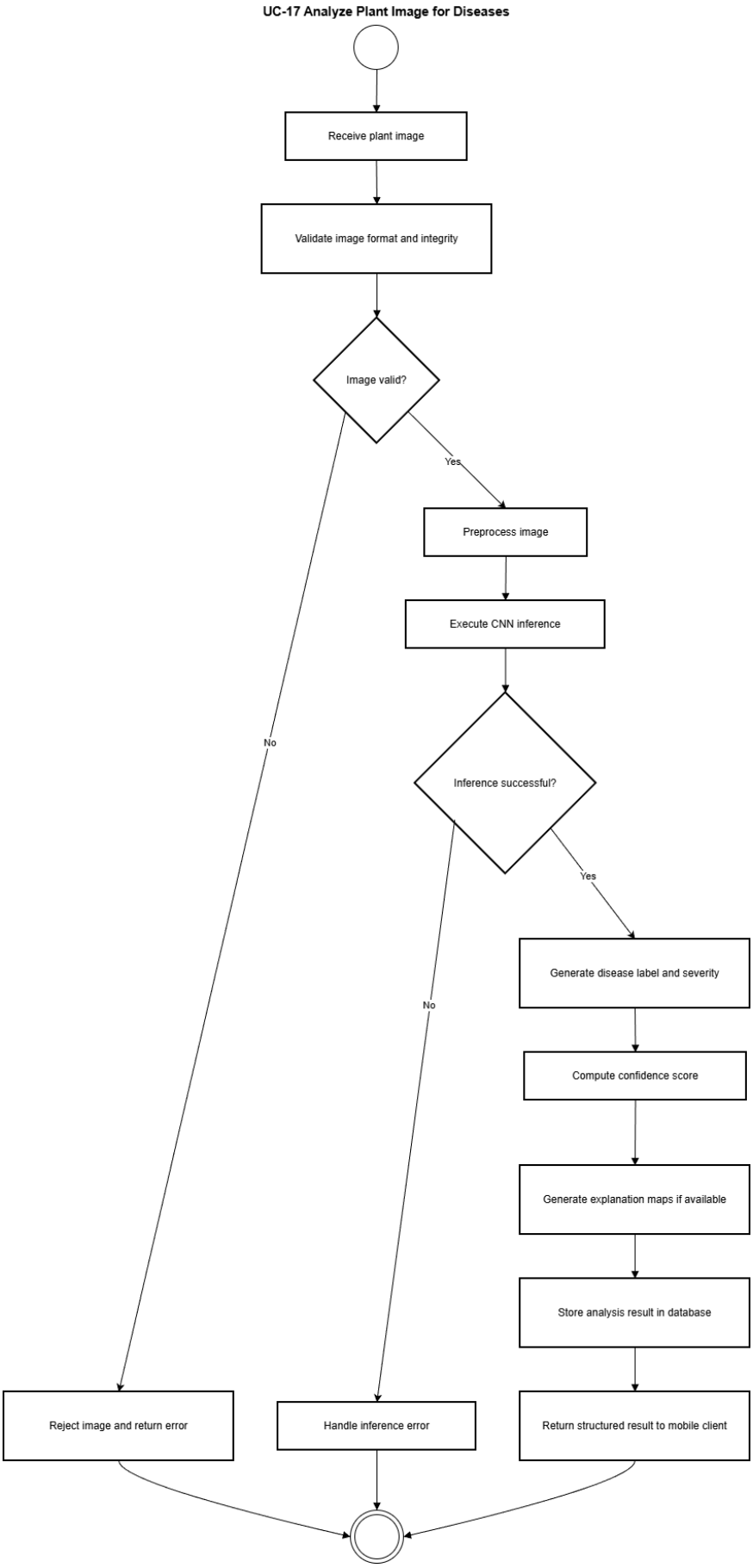

UC 12 View/Export Timetable
User opens Timetable page
System retrieves timetable data
Timetable data available?
Yes
No
Display timetable
Show no timetable data message
User wants to export?
No
Yes
System generates timetable file
User downloads exported timetable


UC 13 View/Export Carbon Footprint Report
User opens Carbon Footprint Reports page
User selects field farm or reporting scope
User selects reporting period
System retrieves carbon footprint data
Data available?
No
Yes
System processes carbon footprint metrics
Display charts tables and KPI values
User wants to export?
No
Yes
User selects CSV or Excel format
System generates carbon report file
Export successful?
Yes
No
Show no carbon footprint data message
User downloads report
Show export failed message

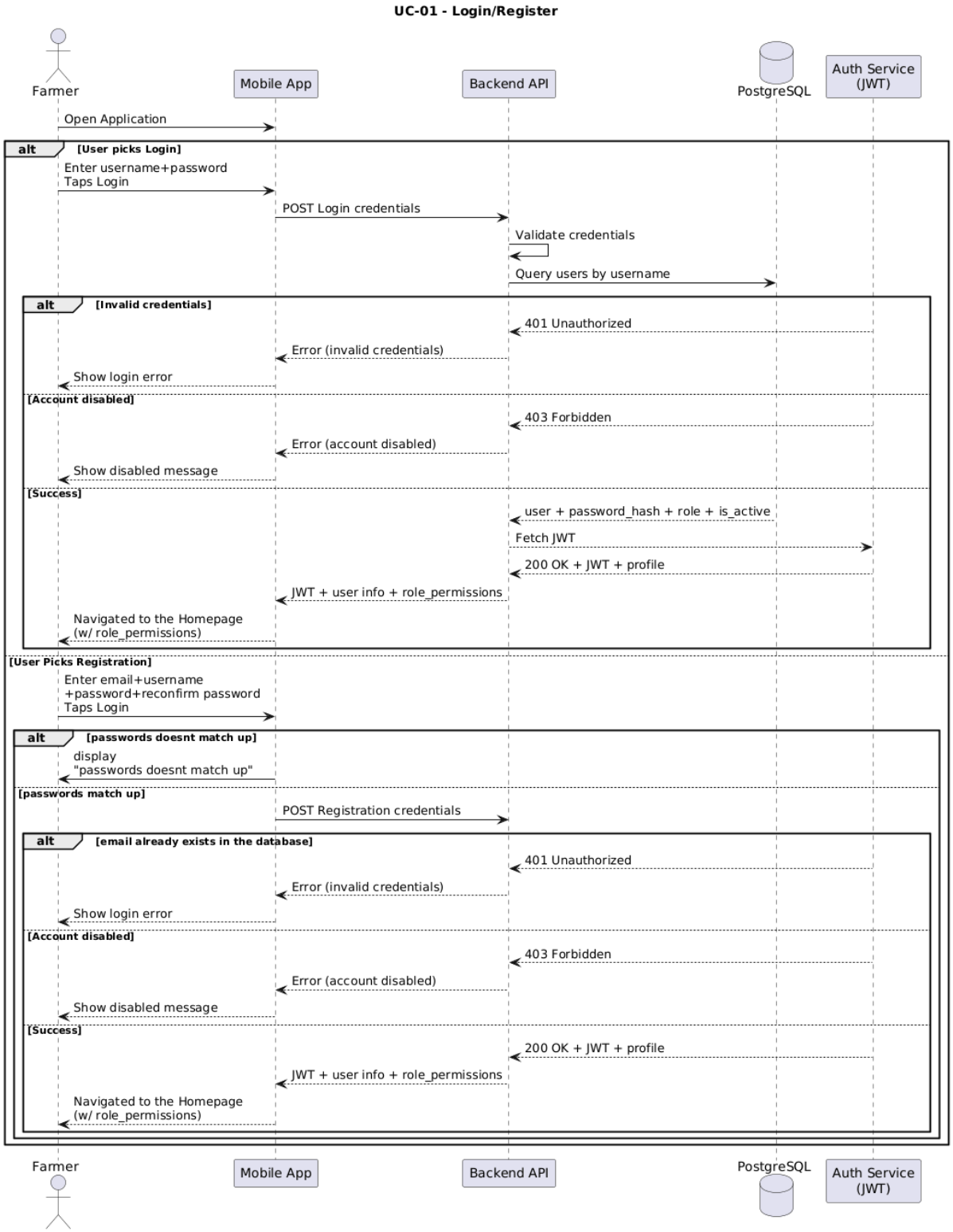

UC 14 Collect and Transmit Sensor Data
Sensor node collects measurements
Send telemetry to gateway
Gateway forwards data to backend
Backend validates node and field
Data valid?
Yes
No
Store measurements in database
Reject data and log error
Update digital twin and irrigation inputs

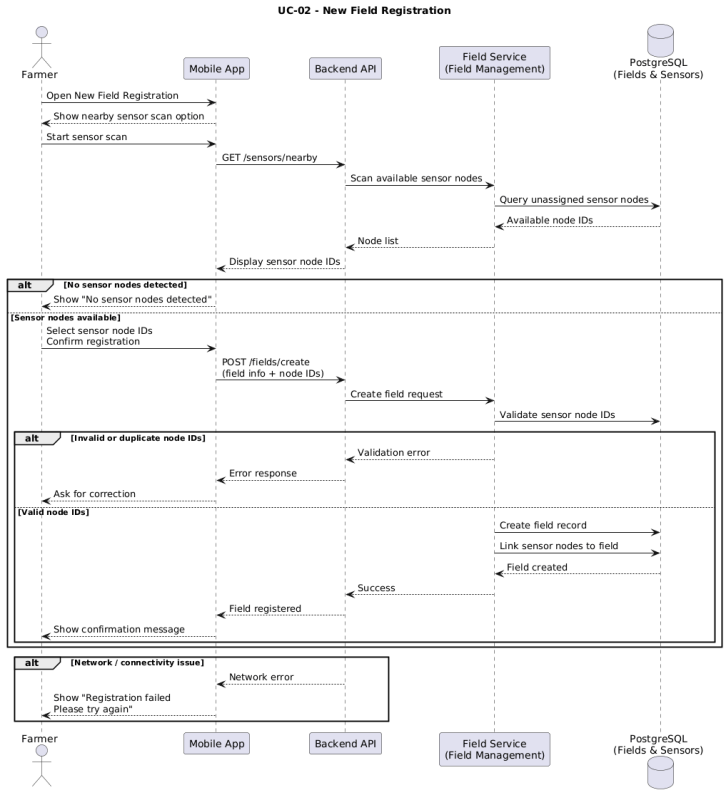

UC 15 Sync Field Data to Backend
New field data becomes available
System prepares data for backend sync
Connectivity available?
No
Yes
Store data locally
Retry synchronization later
Sync failed repeatedly?
No
Yes
Show sync warning or status notification
Send field data to backend
Backend stores data in database
Update related field records
Make updated data available for dashboard and analysis

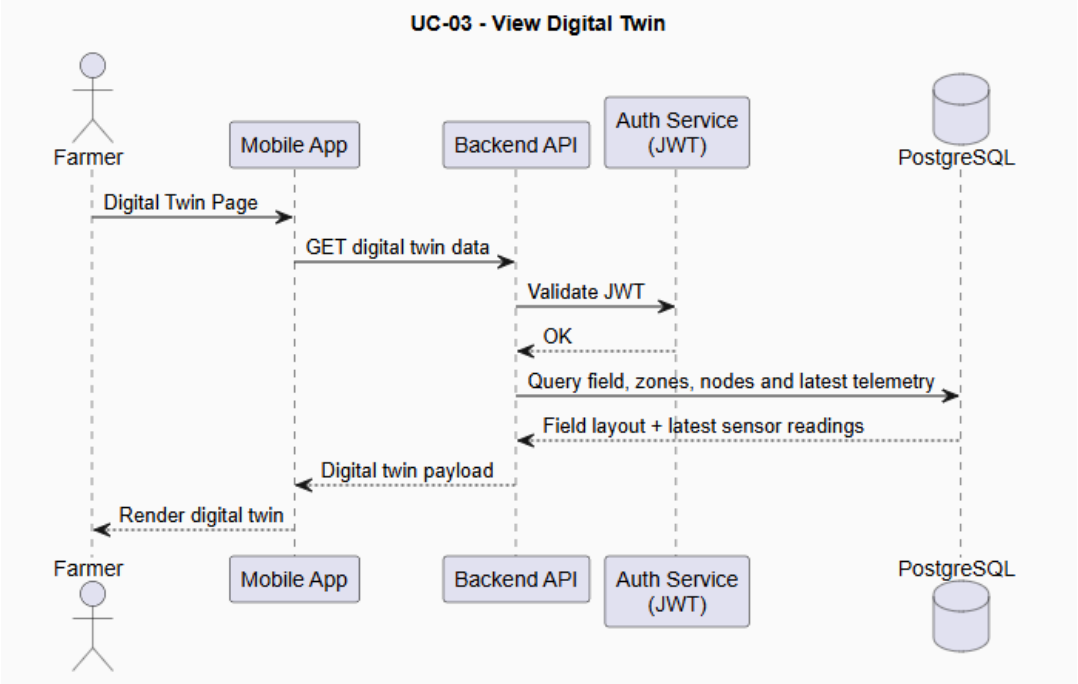

UC 16 Run İrrigation Analysis
System selects field for irrigation analysis
Retrieve sensor readings crop data and irrigation history
Preprocess irrigation data
Data sufficient?
No
Yes
Mark analysis as unavailable
Log missing or invalid data reason
Run rule based irrigation analysis
Calculate irrigation need
Estimate water amount or irrigation duration
Store analysis result in backend
Make result available for irrigation recommendation

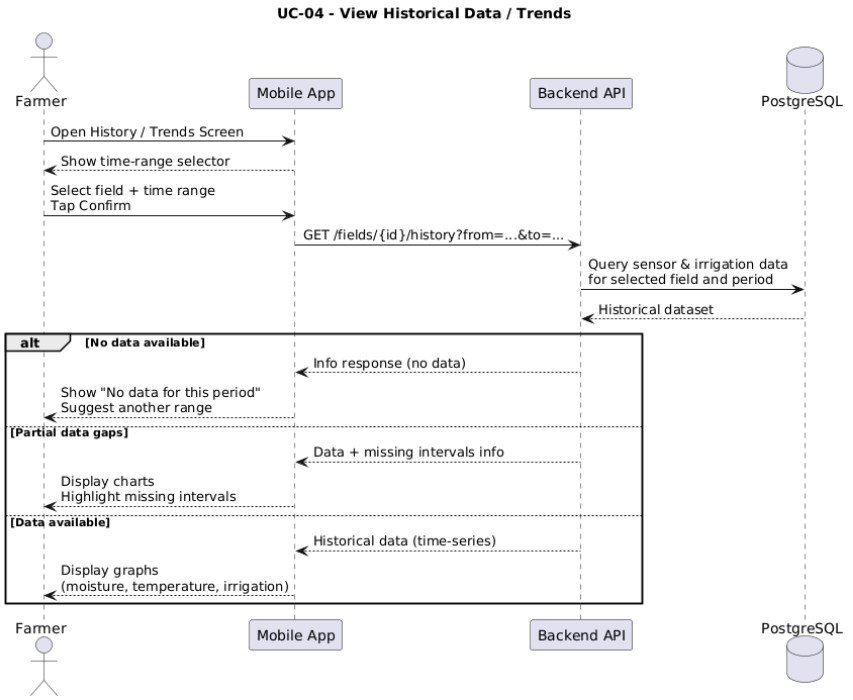

UC-17 Analyze Plant Image for Diseases
Receive plant image
Validate image format and integrity
Image valid?
Yes
No
Preprocess image
Execute CNN inference
Inference successful?
Yes
No
Generate disease label and severity
Compute confidence score
Generate explanation maps if available
Store analysis result in database
Return structured result to mobile client
Handle inference error
Reject image and return error

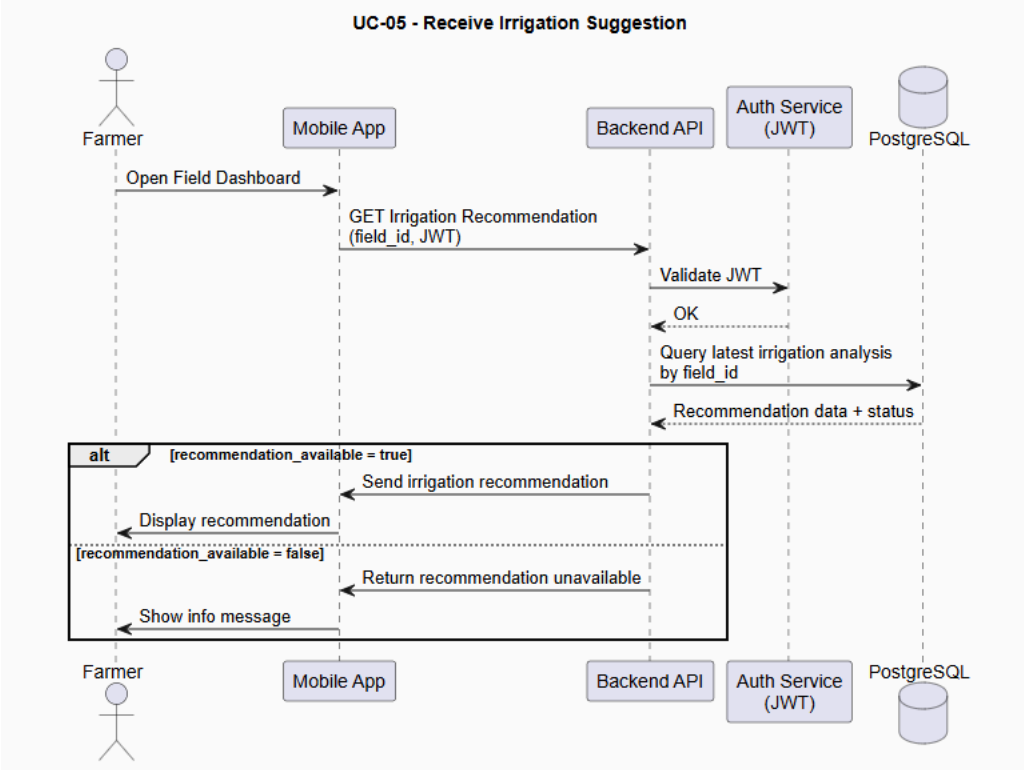
3.3.2. Sequence Diagrams



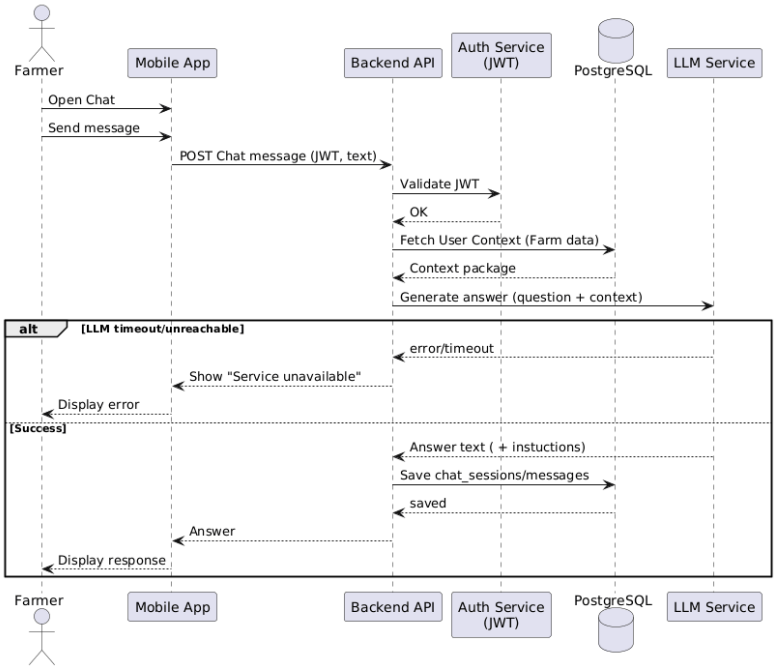

UC-02 - New Field Registration
Farmer
Mobile App
Backend API
Field Service (Field Management)
PostgreSQL (Fields & Sensors)
Open New Field Registration
Show nearby sensor scan option
Start sensor scan
GET /sensors/nearby
Scan available sensor nodes
Query unassigned sensor nodes
Available node IDs
Node list
Display sensor node IDs
alt [No sensor nodes detected]
Show "No sensor nodes detected"
[Sensor nodes available]
Select sensor node IDs
Confirm registration
POST /fields/create (field info + node IDs)
Create field request
Validate sensor node IDs
alt [Invalid or duplicate node IDs]
Validation error
Error response
Ask for correction
[Valid node IDs]
Create field record
Link sensor nodes to field
Field created
Success
Field registered
Show confirmation message
alt [Network / connectivity issue]
Network error
Show "Registration failed Please try again"
Farmer
Mobile App
Backend API
Field Service (Field Management)
PostgreSQL (Fields & Sensors)


UC-03 - View Digital Twin
Farmer
Mobile App
Backend API
Auth Service (JWT)
PostgreSQL
Digital Twin Page
GET digital twin data
Validate JWT
OK
Query field, zones, nodes and latest telemetry
Field layout + latest sensor readings
Digital twin payload
Render digital twin
Farmer
Mobile App
Backend API
Auth Service (JWT)
PostgreSQL

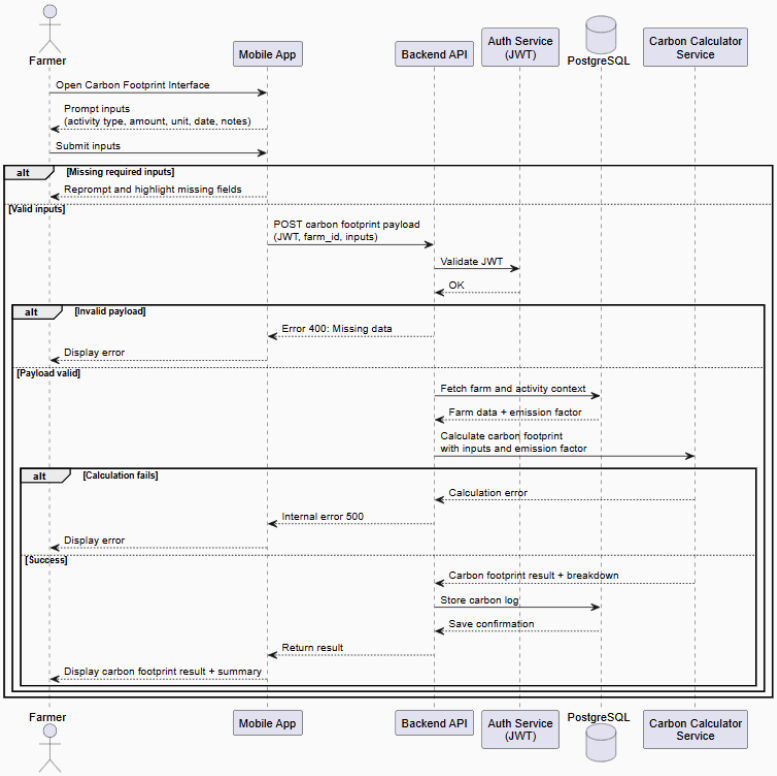

UC-04 - View Historical Data / Trends
Farmer
Mobile App
Backend API
PostgreSQL
Open History / Trends Screen
Show time-range selector
Select field + time range
Tap Confirm
GET /fields/{id}/history?from=...&to=...
Query sensor & irrigation data
for selected field and period
Historical dataset
alt
[No data available]
Info response (no data)
Show "No data for this period"
Suggest another range
[Partial data gaps]
Data + missing intervals info
Display charts
Highlight missing intervals
[Data available]
Historical data (time-series)
Display graphs
(moisture, temperature, irrigation)
Farmer
Mobile App
Backend API
PostgreSQL


UC-05 - Receive Irrigation Suggestion
Farmer
Mobile App
Backend API
Auth Service
(JWT)
PostgreSQL
Open Field Dashboard
GET Irrigation Recommendation
(field_id, JWT)
Validate JWT
OK
Query latest irrigation analysis
by field_id
Recommendation data + status
alt
[recommendation_available = true]
Send irrigation recommendation
Display recommendation
[recommendation_available = false]
Return recommendation unavailable
Show info message
Farmer
Mobile App
Backend API
Auth Service
(JWT)
PostgreSQL

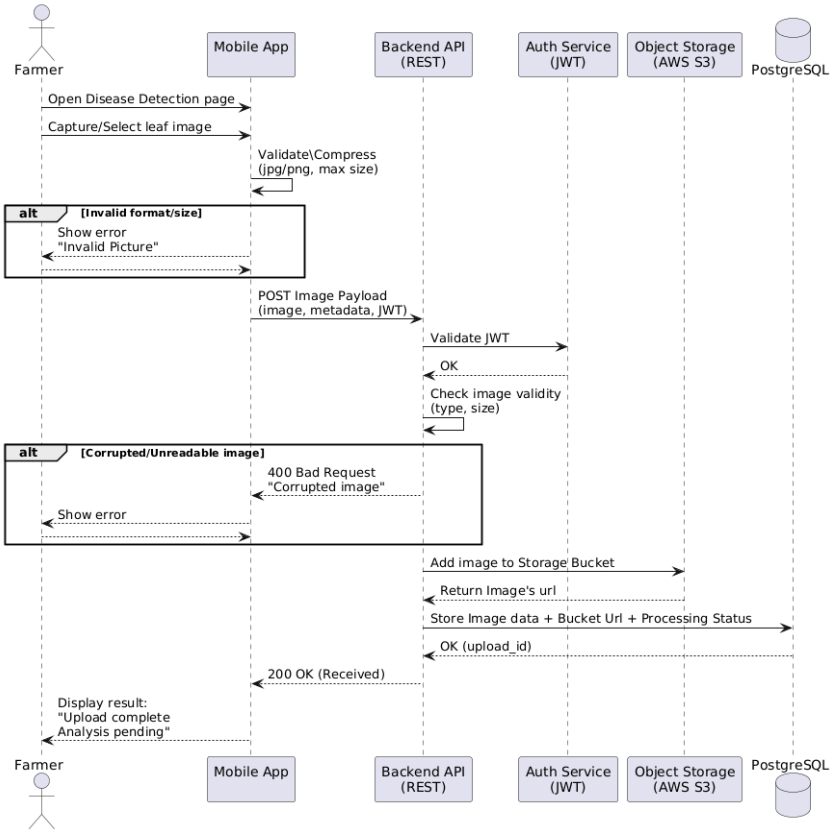
UC-06 - AI Decision-Support Chatbot



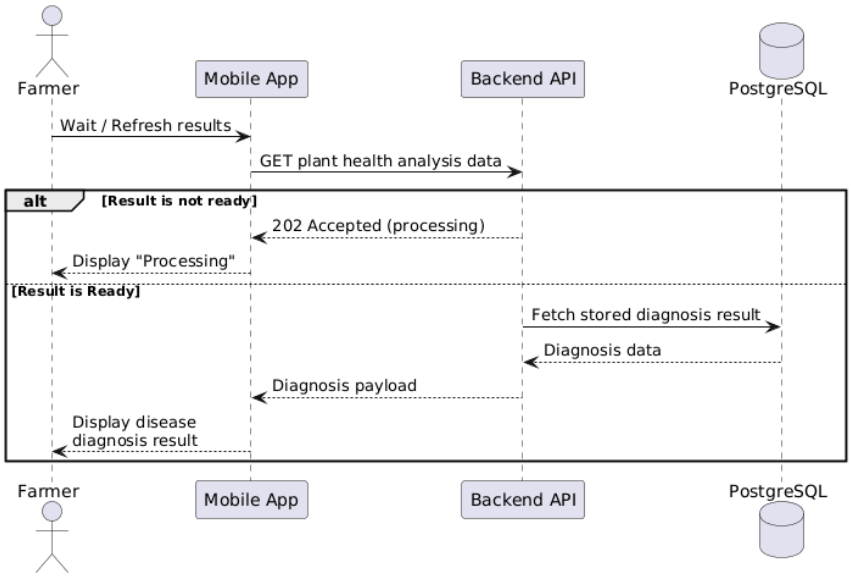
UC-07 - Calculate Carbon Footprint



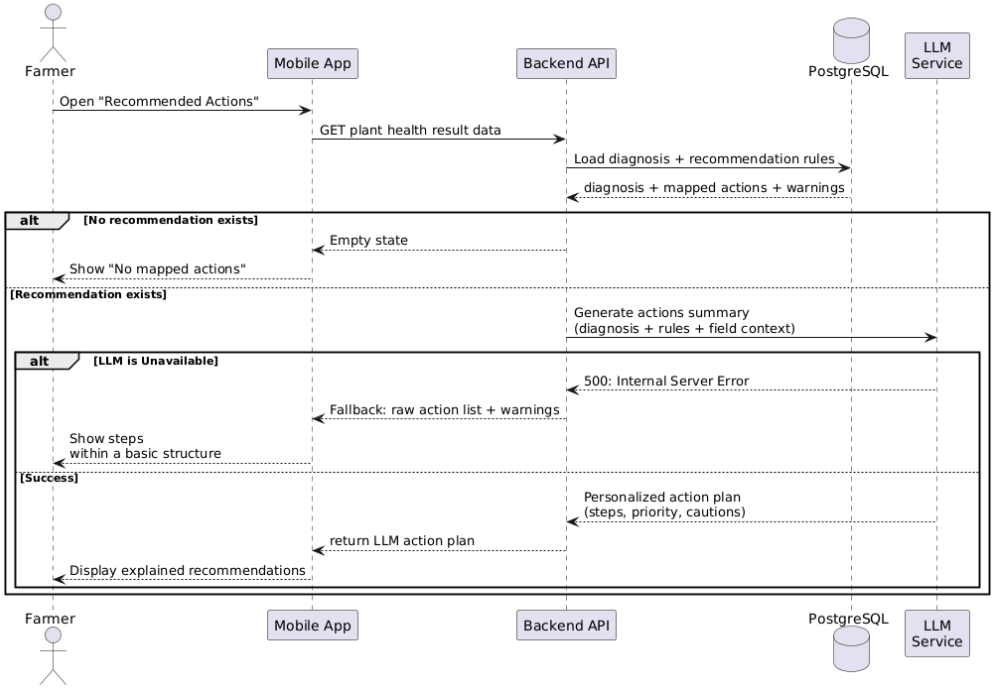
UC-08 - Upload Plant Image



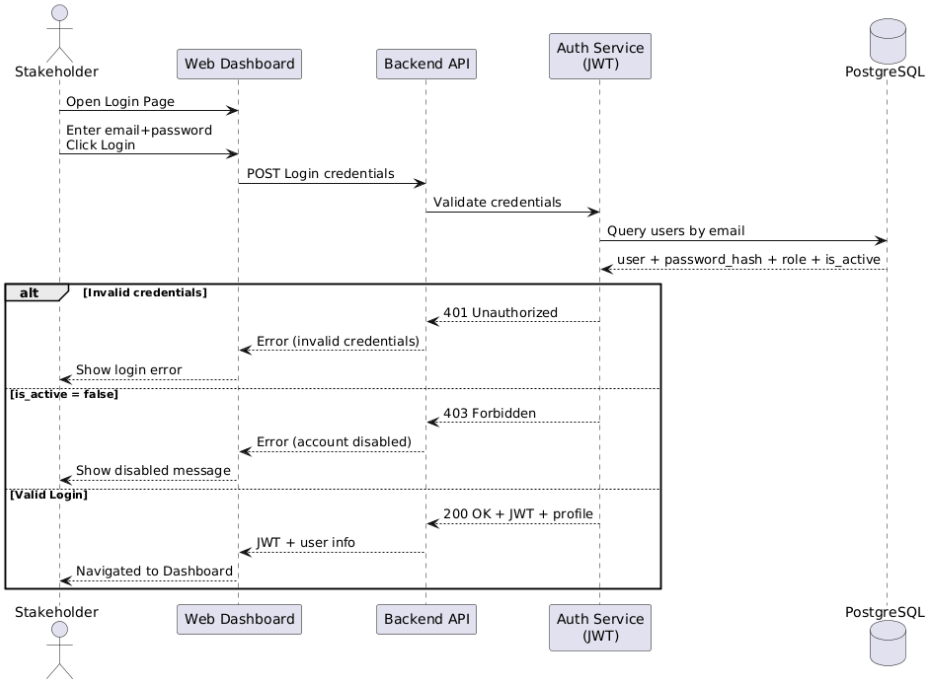
UC-09 - Receive Disease Diagnosis

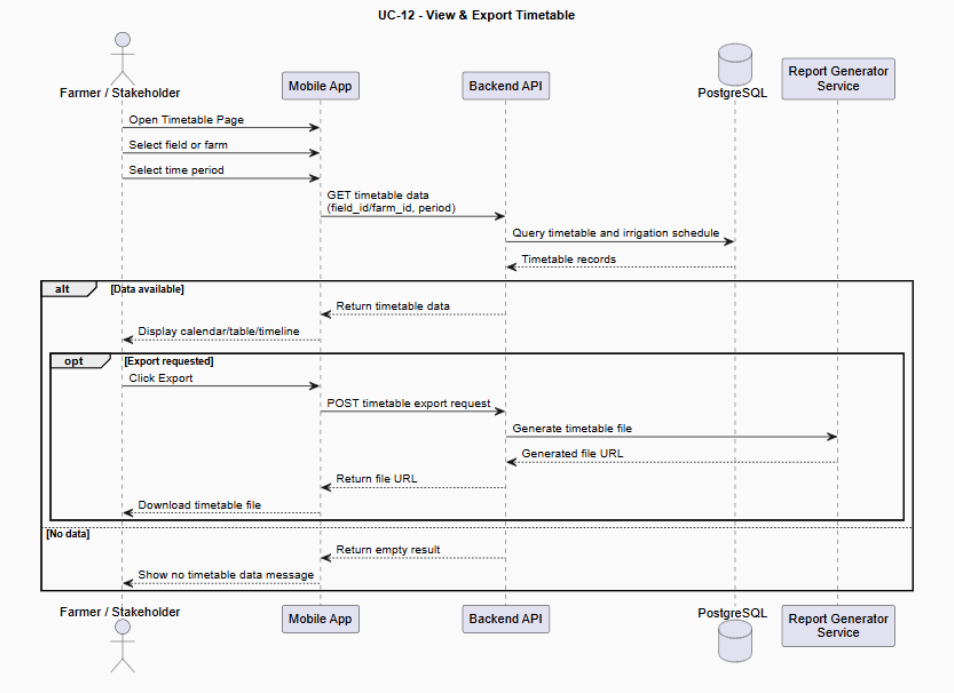


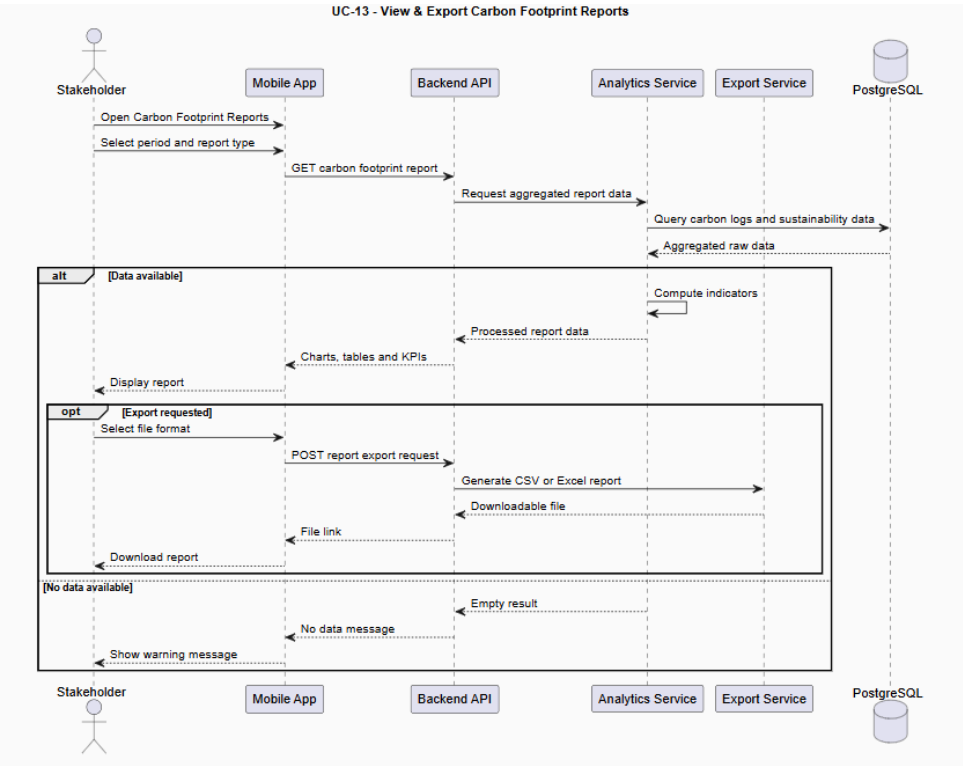
UC-10 - View Recommended Actions

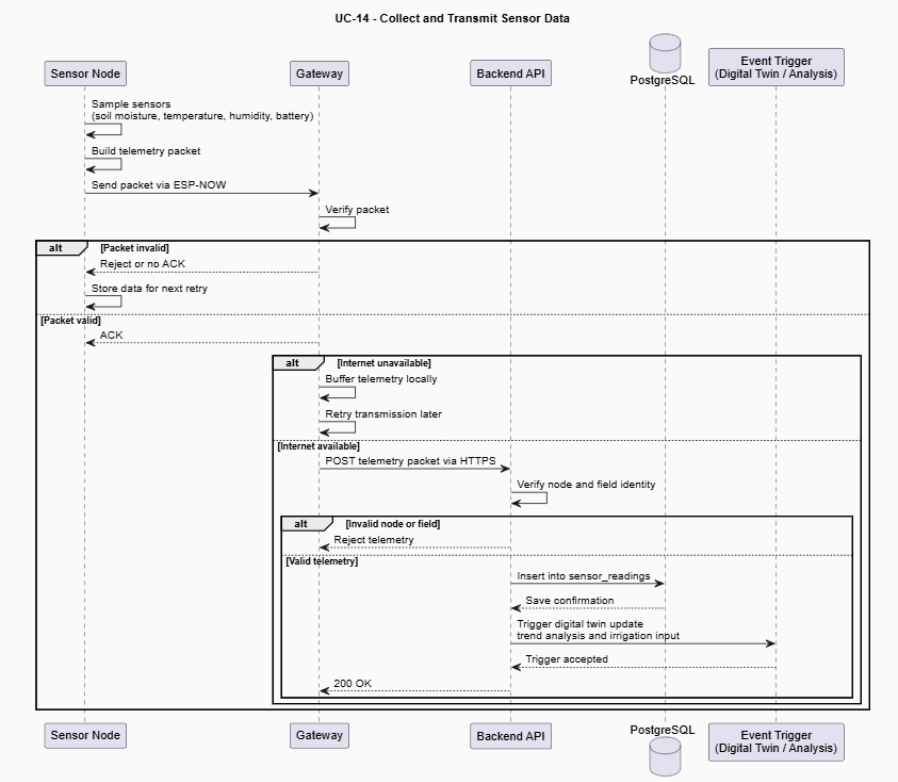


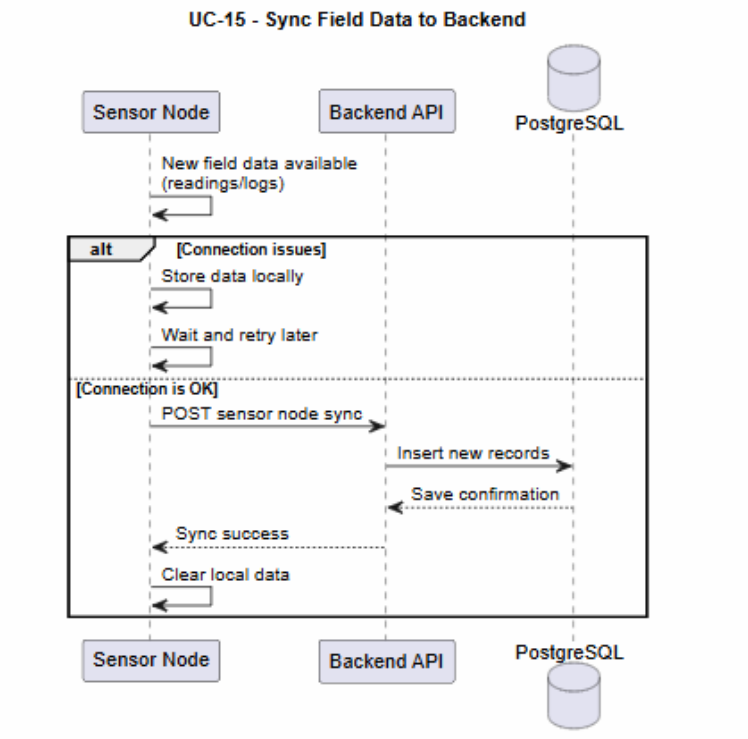
UC-11 - Stakeholder Login

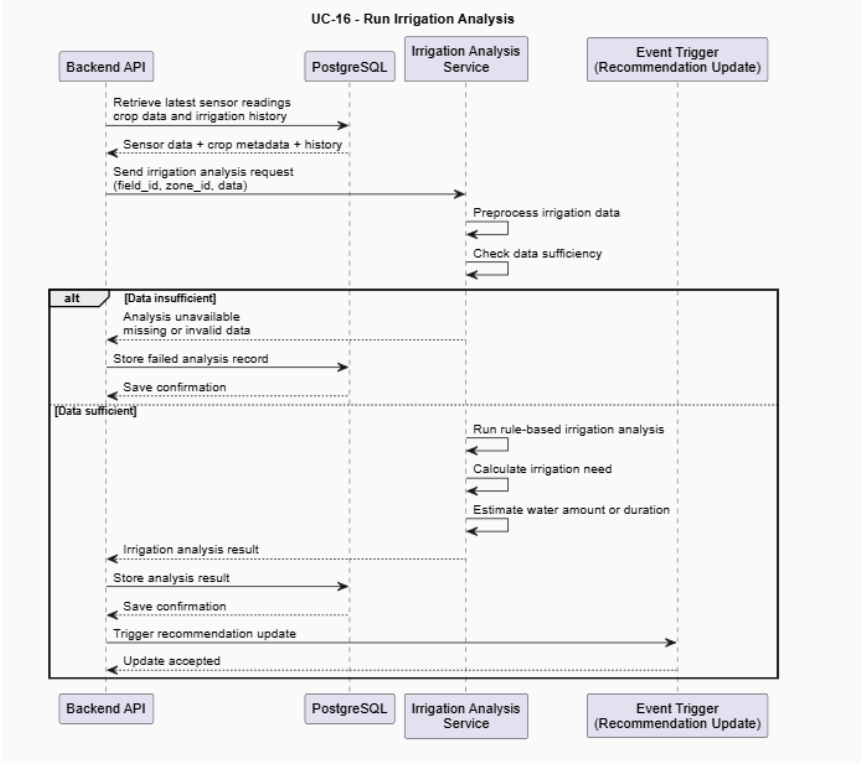


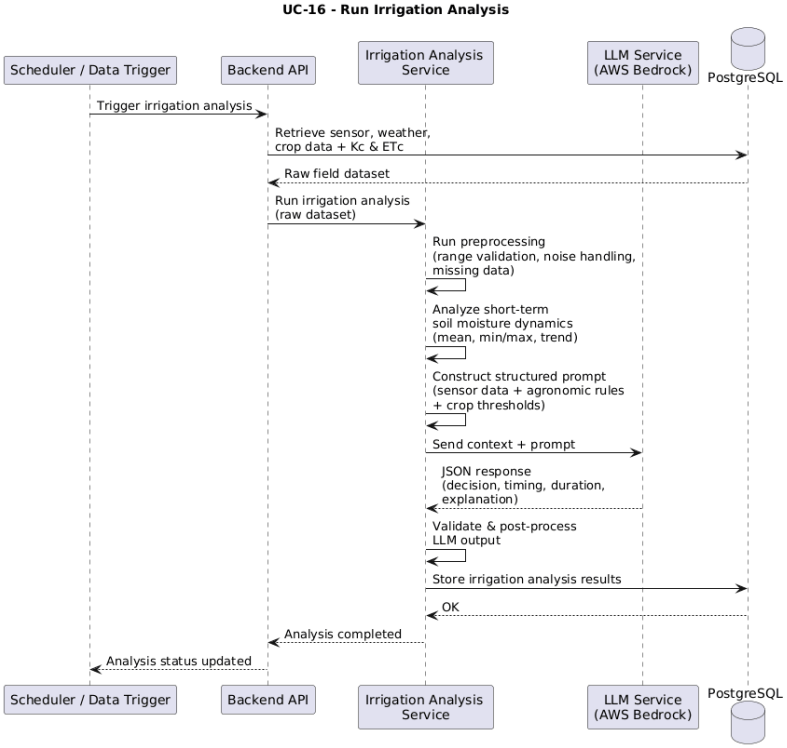

UC-12 - View & Export Timetable
Farmer / Stakeholder
Mobile App
Backend API
PostgreSQL
Report Generator Service
Open Timetable Page
Select field or farm
Select time period
GET timetable data (field_id/farm_id, period)
Query timetable and irrigation schedule
Timetable records
alt
[Data available]
Return timetable data
Display calendar/table/timeline
opt
[Export requested]
Click Export
POST timetable export request
Generate timetable file
Generated file URL
Return file URL
Download timetable file
[No data]
Return empty result
Show no timetable data message
Farmer / Stakeholder
Mobile App
Backend API
PostgreSQL
Report Generator Service

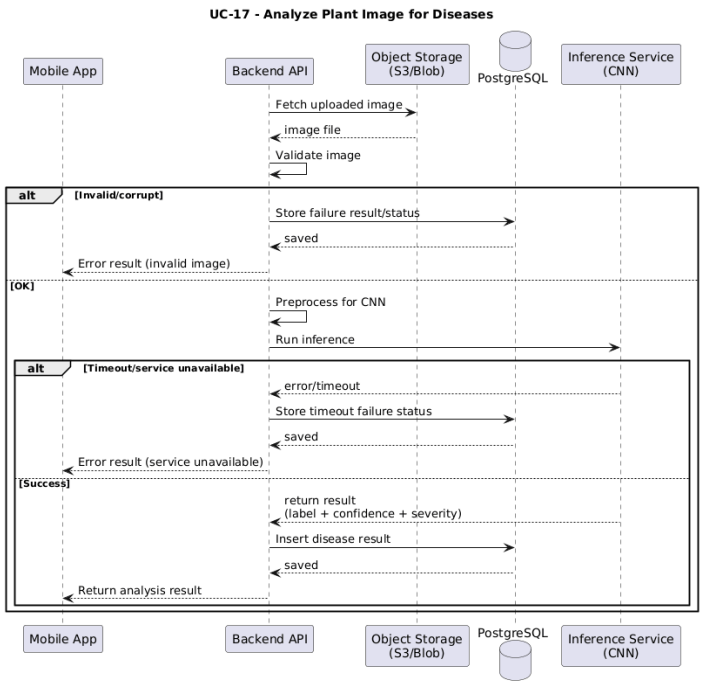

UC-13 - View & Export Carbon Footprint Reports
Stakeholder
Mobile App
Backend API
Analytics Service
Export Service
PostgreSQL
Open Carbon Footprint Reports
Select period and report type
GET carbon footprint report
Request aggregated report data
Query carbon logs and sustainability data
Aggregated raw data
alt
[Data available]
Compute indicators
Processed report data
Charts, tables and KPIs
Display report
opt
[Export requested]
Select file format
POST report export request
Generate CSV or Excel report
Downloadable file
File link
Download report
[No data available]
Empty result
No data message
Show warning message
Stakeholder
Mobile App
Backend API
Analytics Service
Export Service
PostgreSQL


UC-14 - Collect and Transmit Sensor Data
Sensor Node
Gateway
Backend API
PostgreSQL
Event Trigger (Digital Twin / Analysis)
Sample sensors (soil moisture, temperature, humidity, battery)
Build telemetry packet
Send packet via ESP-NOW
Verify packet
alt
[Packet invalid]
Reject or no ACK
Store data for next retry
[Packet valid]
ACK
alt
[Internet unavailable]
Buffer telemetry locally
Retry transmission later
[Internet available]
POST telemetry packet via HTTPS
Verify node and field identity
alt
[Invalid node or field]
Reject telemetry
[Valid telemetry]
Insert into sensor_readings
Save confirmation
Trigger digital twin update trend analysis and irrigation input
Trigger accepted
200 OK
Sensor Node
Gateway
Backend API
PostgreSQL
Event Trigger (Digital Twin / Analysis)


UC-15 - Sync Field Data to Backend
Sensor Node
Backend API
PostgreSQL
New field data available (readings/logs)
alt
[Connection issues]
Store data locally
Wait and retry later
[Connection is OK]
POST sensor node sync
Insert new records
Save confirmation
Sync success
Clear local data
Sensor Node
Backend API
PostgreSQL


UC-16 - Run Irrigation Analysis
Backend API
PostgreSQL
Irrigation Analysis Service
Event Trigger (Recommendation Update)
Retrieve latest sensor readings crop data and irrigation history
Sensor data + crop metadata + history
Send irrigation analysis request (field_id, zone_id, data)
Preprocess irrigation data
Check data sufficiency
alt
[Data insufficient]
Analysis unavailable missing or invalid data
Store failed analysis record
Save confirmation
[Data sufficient]
Run rule-based irrigation analysis
Calculate irrigation need
Estimate water amount or duration
Irrigation analysis result
Store analysis result
Save confirmation
Trigger recommendation update
Update accepted
Backend API
PostgreSQL
Irrigation Analysis Service
Event Trigger (Recommendation Update)


UC-16 - Run Irrigation Analysis
Scheduler / Data Trigger
Backend API
Irrigation Analysis Service
LLM Service (AWS Bedrock)
PostgreSQL
Trigger irrigation analysis
Retrieve sensor, weather, crop data + Kc & ETc
Raw field dataset
Run irrigation analysis (raw dataset)
Run preprocessing (range validation, noise handling, missing data)
Analyze short-term soil moisture dynamics (mean, min/max, trend)
Construct structured prompt (sensor data + agronomic rules + crop thresholds)
Send context + prompt
JSON response (decision, timing, duration, explanation)
Validate & post-process LLM output
Store irrigation analysis results
OK
Analysis completed
Analysis status updated
Scheduler / Data Trigger
Backend API
Irrigation Analysis Service
LLM Service (AWS Bedrock)
PostgreSQL



4. Methodology

4.1. Irrigation Recommendation

TARAS does not use a fully data-driven ML model for irrigation recommendation. Instead, it uses an agronomy-based rule engine supported by real-time sensor data and zone-specific calibration values.

The backend evaluates soil moisture, temperature, humidity, crop type, environment type, irrigation method, and growth stage. For tomato, growth-stage-based moisture thresholds determine whether irrigation is needed. In greenhouse zones, TARAS estimates irrigation duration using Kc values, simplified ET0, soil moisture deficit, and irrigation calibration. In pot-based manual zones, it calculates the required water amount in milliliters.

The output includes irrigation need, timing, duration or water volume, urgency level, predicted soil moisture after irrigation, follow-up check time, and reasoning. The LLM is used as an advisory layer to explain recommendations, while the actual decision logic remains rule-based and transparent.

4.1.1. Problem Formulation

The irrigation recommendation problem in TARAS is formulated as a rule-guided decision-making task. It determines whether irrigation is needed by evaluating real-time sensor data, crop characteristics, environment type, irrigation method, and plant growth stage.

4.1.2. Data Acquisition and Preprocessing

In TARAS, irrigation decisions use structured data collected from sensors, zone metadata, and system calibration parameters.

The main input is real-time field sensor data, especially soil moisture percentage. Air temperature and humidity are also used to represent current environmental conditions. For each zone, TARAS uses the latest valid sensor readings and averages data from available sensor nodes when needed.

Crop and environment metadata include crop type, environment type, irrigation method, planting date, and growth stage. If the growth stage is not manually provided, it is estimated from the planting date. System parameters include growth-stage-based soil moisture thresholds, Kc values, irrigation gain values, pot calibration parameters, safety limits, and default follow-up check time.

Historical irrigation data are also used for calibration. Previous irrigation actions and follow-up soil moisture measurements help estimate how much soil moisture increases after a given water amount or irrigation duration.

Before generating a recommendation, the backend performs lightweight validation. It checks whether required values such as soil moisture, temperature, humidity, crop name, environment type, irrigation mode, and growth stage are available. It also applies safety limits to prevent excessive irrigation. These steps ensure that the rule-based irrigation engine receives consistent and reliable inputs while keeping the process transparent and explainable.

4.1.3. Algorithmic Approach

The irrigation recommendation process in TARAS follows a rule-guided backend workflow.

First, the system aggregates real-time sensor inputs such as soil moisture, temperature, and humidity for each zone. It also retrieves crop information, environment type, irrigation method, planting data, growth stage, and calibration parameters.

Next, the backend determines the plant growth stage and compares the current soil moisture value with growth-stage-based thresholds. If the zone is a greenhouse, TARAS estimates irrigation duration using Kc values, simplified ET0, soil moisture deficit, and irrigation gain calibration[6]. If the zone uses pot-based manual irrigation,

it calculates the required water amount in milliliters using soil moisture deficit and pot calibration values.

The system then applies safety limits, estimates the predicted soil moisture after irrigation, sets urgency level, and defines a follow-up check time. The final output is stored as a structured irrigation recommendation including decision, timing, duration or water amount, predicted result, urgency, and reasoning.

4.1.4. System Evaluation Metrics

TARAS evaluates irrigation performance using operational and decision-oriented metrics instead of traditional ML metrics.

The main evaluation criterion is whether irrigation recommendations help soil moisture move toward the target range after irrigation. Threshold compliance is also monitored by checking whether soil moisture stays within growth-stage-based optimal and critical limits.

The system evaluates over-irrigation and under-irrigation by tracking cases where soil moisture rises above the recommended range or drops below critical thresholds. Recommendation stability is assessed by observing whether similar zone conditions produce consistent decisions over time.

Historical irrigation records and follow-up measurements are used for validation and calibration. By comparing recommended water amount or duration with the actual soil moisture change after irrigation, TARAS can improve zone-specific calibration values and make future recommendations more consistent.

4.2. Disease Detection

4.2.1. Problem Formulation

This module is designed as an image processing and ML component of the TARAS system, focusing on the analysis of plant leaf images for disease identification. The primary objective is to enable users to capture or upload a leaf image, after which the system automatically detects the presence of plant disease and classifies its type.

Formally, the task is modeled as a multi-class image classification problem. Each input image **X** is mapped to a disease label **Y**, selected from a predefined set of categories representing both healthy and diseased plant conditions. The model aims to achieve high classification

accuracy while ensuring robust generalization performance on previously unseen data.

4.2.2. Data Acquisition and Preprocessing

4.2.2.1. Data Acquisition

Plant leaf images used in this study are primarily obtained from the publicly available PlantVillage dataset, which provides a large collection of labeled images under controlled conditions[1]. To improve real-world applicability and model generalization, an additional dataset was constructed to better represent field conditions. This supplementary dataset was collected using a combination of automated image extraction tools (implemented in Python) and manual collection processes.

By combining the structured and balanced nature of the PlantVillage dataset with the variability of real-world images, the overall dataset aims to better capture practical conditions encountered in real deployments.

4.2.2.2. Preprocessing

Images are resized so the shorter side is 256 pixels and then centre-cropped to the model input of 224 × 224. Pixel values are normalized with the standard ImageNet mean and standard deviation to match the ImageNet-pretrained backbones.

To improve generalization, especially due to the domain gap between the controlled PlantVillage dataset and real-world images, data augmentation techniques are applied. These include random rotations, horizontal flips, slight zooming, and brightness variations, simulating diverse field conditions such as different lighting, orientations, and capture angles. This preprocessing pipeline ensures both computational efficiency and robustness under real-world deployment scenarios.

4.2.3. Algorithmic Approach

The disease detection module uses a deep-learning image-classification approach. On the server, classification is performed by an ensemble of modern convolutional and transformer backbones (ConvNeXtV2-Base [7], [2] and two Swin-Transformer models [3]); a compact EfficientNet-B0 model [5], distilled from this ensemble [4], runs on-device for low-latency inference.

Preprocessed images are passed through the CNN backbone, which extracts high-level visual features such as texture, color, and

structural patterns. These features enable the model to distinguish between healthy and diseased leaves, as well as different disease categories.

A pretrained model is used to leverage general visual knowledge from large-scale datasets, improving robustness under varying real-world conditions such as lighting changes, background variability, and leaf orientation.

The final prediction is generated through a classification layer that maps extracted features to predefined disease classes, selecting the label with the highest probability.

4.2.4. Model Selection and Comparative Evaluation

Several backbone families were considered, spanning lightweight CNNs (MobileNetV3, EfficientNet-B0), deeper CNNs (ResNet, EfficientNet), and modern convolutional/transformer architectures (ConvNeXt, Swin).

The deployed solution combines a high-accuracy server-side ensemble (ConvNeXtV2-Base + Swin-Small + Swin-Base, equal-weight softmax average) with a distilled EfficientNet-B0 student for on-device use, balancing accuracy against latency and model size.

4.2.5. System Evaluation Metrics

Model performance is to be evaluated using multiple complementary metrics to capture different aspects of classification quality:

- **Accuracy**: Overall proportion of correctly classified images
- **Precision**: Ability of the model to avoid false disease predictions
- **Recall**: Ability of the model to correctly identify diseased samples
- **F1-score**: Harmonic mean of precision and recall

4.2.6. Implementation Details

The disease detection module is implemented in PyTorch (using the timm model library) and trained offline. For deployment it is exported in two forms: the server ensemble is served on AWS Lambda (PyTorch, with an ONNX-Runtime variant).

The distilled EfficientNet-B0 student is exported to TensorFlow Lite (fp16) for on-device inference in the mobile app. The module classifies a preprocessed leaf image and outputs the most probable disease label

among the 14 classes; its modular structure allows independent model updates without affecting the rest of the system.

4.3. LLM-Based Advisory & Decision Support

4.3.1. Problem Formulation

The LLM-based advisory component aims to translate complex agricultural data into clear, context-aware decision support. While the system's predictive modules output raw numbers and categorical labels (such as disease types or moisture levels), farmers often need these results explained in plain language. Therefore, we formulate this as a natural language reasoning task: transforming structured system data into actionable, easy-to-understand agronomic recommendations.

To act as an intelligent bridge between the raw data and the user, the language model processes two main inputs: the real-time system context (soil moisture, irrigation thresholds, disease status) and the user's direct query. By combining these, it turns raw metrics into practical advice.

4.3.2. Input Representation and Context Construction

The LLM needs a well-structured input to provide accurate answers. We build this context by combining irrigation predictions, disease detections, and current environmental data.

To prevent the model from confusing background system data with the user's chat message, we isolate the information. The assistant is given a cached system prompt that defines its role, brevity rules, a condensed agronomic reference, and a set of callable tools. Rather than receiving all field data up front, the model is provided only a lightweight inventory of the user’s farms, fields, zones (with their identifiers) including the chat history, and is instructed to fetch the specific data it needs, current sensor values, zone thresholds, irrigation history, carbon summary, disease history, active alerts, by calling the corresponding tools on demand. This keeps each request focused and avoids dumping unprompted technical reports into the conversation. This targeted approach prevents the assistant from giving unprompted, robotic technical reports and keeps the conversation natural and relevant to the farmer's needs.

4.3.3. Response Generation and Explainability

The assistant generates actionable advice by explaining the system's findings in everyday language. It explicitly references key data points, such as current soil moisture or visual disease symptoms, to back up its statements. This approach builds user trust because it goes beyond

simply telling the farmer *what* to do; it explains *why* the recommendation is being made based on their specific field conditions.

4.3.4. System Evaluation and Limitations

Because the LLM acts as an interpreter rather than a prediction engine, evaluating it focuses on qualitative metrics: how closely it sticks to the provided data, how logical its answers are, and how useful the advice is to the farmer. The main goal is to maintain "zero-hallucination" boundaries—ensuring the model never invents facts.

However, the system has clear limitations. The assistant is only as reliable as the data it receives. If the IoT sensors or camera modules provide incorrect readings, the LLM will base its advice on that flawed data. Additionally, while the strict prompt prevents the AI from making things up, it also means the system might struggle to provide advice on situations it hasn't been programmed to handle, such as entirely new crop types or undocumented diseases. Therefore, the assistant is designed to be a helpful decision-support tool, engineered to assist—not replace—the judgment of expert agronomists.

4.3.5. Implementation Details

The advisory module is implemented as a service inside the TARAS Node.js/Express backend and is reached from the mobile app over a REST endpoint. On each request the backend builds the per-request context (the farm/field/zone inventory and recent history) and runs an agentic tool-calling loop with the language model: the model may issue one or more tool calls, each of which the backend executes against the PostgreSQL database via Prisma (live reads of sensor metrics, thresholds, irrigation jobs, alerts, etc.) before returning the results to the model; the loop continues until the model produces its final natural-language answer, which is returned to the app. The current LLM provider used is Anthropic’s API with prompt caching on the system prompt.

4.4. Sustainability & Carbon Footprint Calculation

4.4.1. Problem Formulation

This module calculates the carbon footprint of agricultural activities in the farm. The goal is to estimate how much greenhouse gas is produced during operations such as irrigation, energy use, fuel consumption and fertilizer application.

The problem is defined as converting activity data into carbon emission values. Each activity produces a certain amount of emissions based on its usage and a corresponding emission factor.

The system takes activity inputs and calculates total carbon emissions using a simple rule-based formula stated in 4.4.3 Calculation Approach.

The emission factors and calculation method used in this system are obtained from standardized sources such as IPCC and FAO, ensuring consistency and reliability in the calculations. This approach is transparent and does not rely on machine learning models.

4.4.2. Data Acquisition & Input Representation

The carbon footprint calculation module uses data collected from multiple sources within the system.

The main inputs include:

- Energy consumption data
- Fuel usage data
- Fertilizer application

These inputs are obtained from user inputs. For example, energy or water consumption can be recorded by the user when a bill is received.

In addition to activity data, the system uses emission factors obtained from standardized sources such as IPCC and FAO. These values are stored within the system and matched with the corresponding activity type.

All inputs are converted into a consistent format before calculation. This includes unit standardization and simple validation to ensure values are within reasonable ranges.

This structure allows the system to combine data from different modules and represent them in a unified way for carbon footprint calculation.

4.4.3. Calculation Approach

The carbon footprint is calculated using a simple rule-based approach based on activity data and predefined emission factors. For each activity (such as energy use, fuel consumption or fertilizer application), the system calculates emissions using the following formula:

Carbon Emissions = Activity Amount × Emission Factor

Each activity is processed separately, and the total carbon footprint is obtained by summing all individual emissions. The calculation process follows these steps:

- Activity data is collected from system inputs (user input or logs)
- The corresponding emission factor is selected based on the activity type
- Emissions are calculated for each activity
- All emissions are summed to obtain the total carbon footprint

The emission factors are predefined and stored in the system based on standardized sources such as IPCC and FAO.

4.4.4. Output Metrics & Interpretation

The system presents carbon footprint results in a clear and simple way. The main output is the total carbon emissions (kg CO_2-equivalent), representing the overall environmental impact.

In addition, emissions are shown as a breakdown by activity type (such as, energy, fuel, fertilizer), helping users see which sources contribute the most.

Emissions can also be displayed in normalized forms, such as:

- per area
- per production amount (if available)

This allows users to better understand and compare their environmental impact.

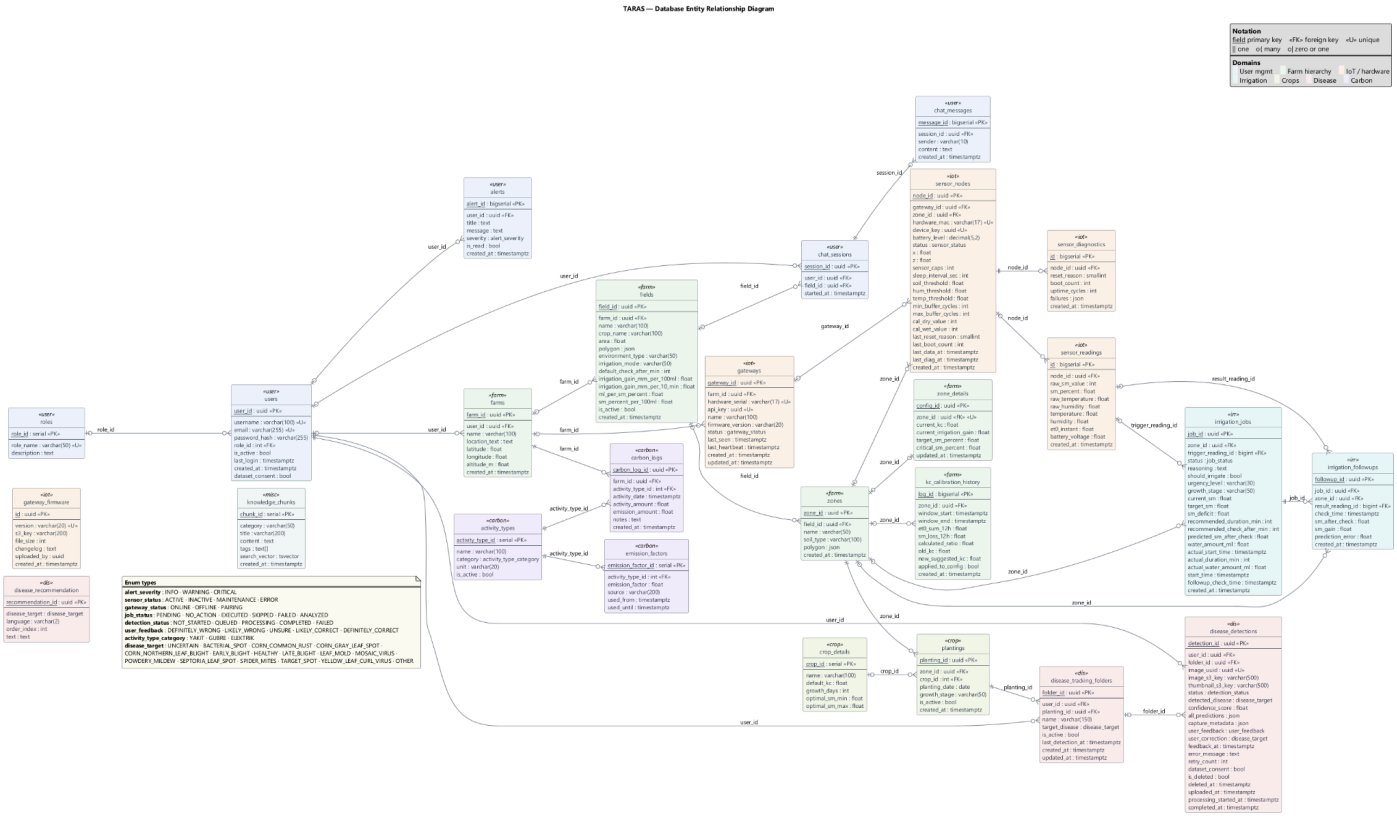
4.4.5. Implementation Details

The carbon footprint calculation is implemented as part of the TARAS backend within the analytics layer. The system uses activity data along with predefined emission factors obtained from sources such as IPCC and FAO. Before calculation, input values are standardized into common units and checked for basic validity

Carbon emissions are calculated using a rule-based formula for each activity, and the results are aggregated to obtain the total footprint. The calculated outputs are then stored and sent to the user interface through backend services, where they are displayed together with other system outputs.

5. Data Design

5.1. Entity-Relationship Diagram



5.2. Data Pipeline

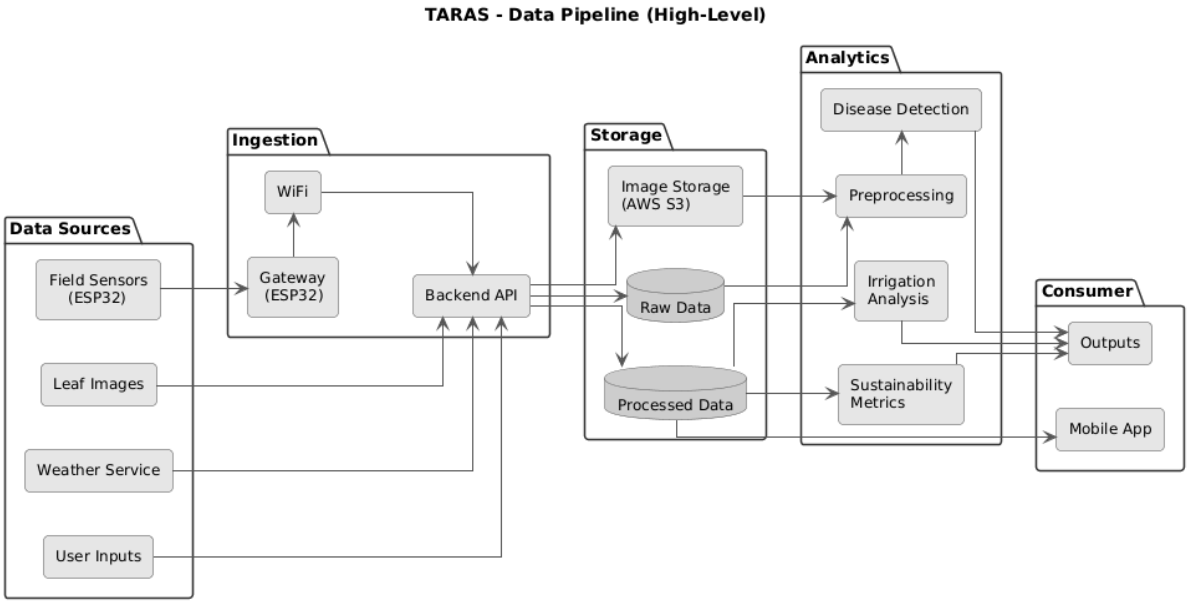
The TARAS data pipeline defines how heterogeneous data collected from field sensors, user inputs, and image uploads are transformed into reliable and analysis-ready information. The pipeline is designed to support real-time monitoring, historical analysis, and decision-support operations while maintaining data consistency and traceability.

Field sensor measurements such as soil moisture and environmental conditions are either transmitted to gateway devices first. In parallel, crop metadata, irrigation parameters and plant images are received through backend interfaces. All incoming data are timestamped and associated with field and device identifiers to ensure coherent integration.

Before storage, the data undergo preprocessing steps including timestamp alignment, unit normalization, basic quality checks, and context-aware

filtering to handle missing values and abnormal sensor readings. Processed numerical data are stored in a centralized database, while image data are stored in dedicated storage buckets with linked metadata.

The processed data are then consumed by analytical modules responsible for irrigation estimation, disease detection, and sustainability analysis. Derived outputs such as irrigation recommendations, classification results and environmental indicators are stored as processed records and delivered to the mobile app through backend APIs. This structured pipeline enables consistent data flow across sensing, analytics and visualization layers and provides a scalable foundation for the TARAS platform.



6. User Interface Design

6.1. Interface Hierarchy

The interface follows a hierarchical structure that prioritizes critical agricultural information. Core metrics such as soil moisture, temperature, and field status are placed at the highest level and presented through large, clearly separated data cards. This allows users to access essential information with minimal navigation and reduces cognitive effort during decision-making.

The hierarchy is intentionally simple to support farmers with different technical backgrounds, especially older users with limited digital experience. By minimizing nested menus and visual clutter, the interface helps users quickly identify actionable insights without feeling overwhelmed.

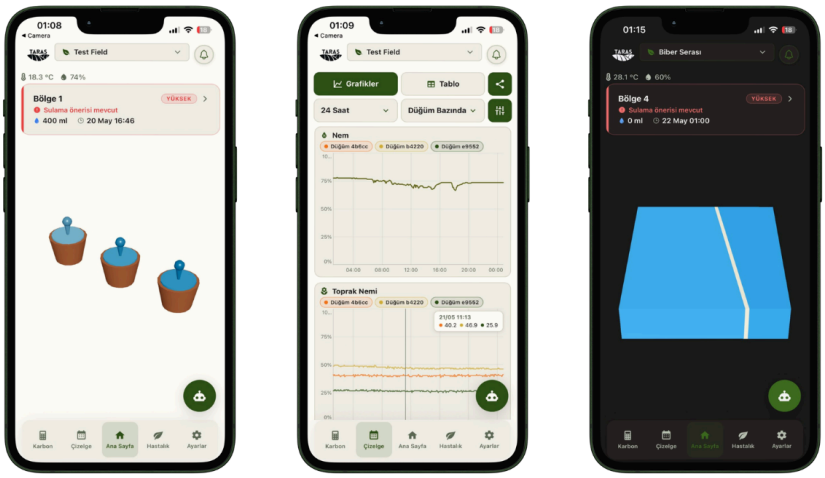
6.2. Visual Presentation

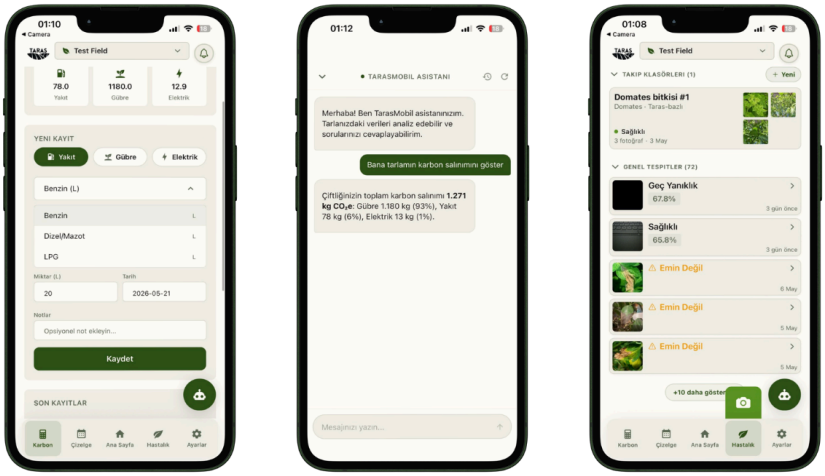
A balanced and minimalistic color palette was adopted for the visualization and interaction layer. The palette is kept simple so that digital twin visualizations remain visually distinct from the interface background. Light and dark modes assign inverse roles to background and text elements, preserving visual separation and comfort under different lighting conditions.

6.3. Accessibility and Responsivity

The interface design adheres to web accessibility and readability principles, including WCAG contrast recommendations, to support usability across varying lighting conditions and display settings. Text sizes, spacing, and contrast levels are specified to compensate for age-related declines in visual acuity and cognitive flexibility.

To further reduce cognitive load, essential information is structured to be presented in a concise and visually separated manner. In addition, an AI-powered conversational assistant is provided as a core interaction mechanism to support users with varying levels of technological literacy [8].





7. Conclusion

This document outlined the system architecture and methodology of the TARAS precision agriculture platform, focusing on its layered structure, modular subsystems and data-driven decision logic. By combining field sensing, agronomy-based irrigation analysis, AI-supported disease detection and user-centered interface design, TARAS provides a scalable and maintainable foundation for intelligent agricultural support. The proposed architecture enables future extensions while ensuring reliable and accessible system operation.

8. References

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using Deep Learning for Image-Based Plant Disease Detection," Frontiers in Plant Science, vol. 7, p. 1419, 2016.
[2] S. Woo, S. Debnath, R. Hu, X. Chen, Z. Liu, I. S. Kweon, and S. Xie, “ConvNeXt V2: Co-designing and Scaling ConvNets with Masked Autoencoders,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), Vancouver, BC, Canada, 2023, pp. 16133–16142.
[3] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows,” in Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), Montreal, QC, Canada, 2021, pp. 10012–10022, doi: 10.1109/ICCV48922.2021.00986.
[4] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in Proc. NIPS Deep Learning and Representation Learning Workshop, 2015.
[5] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in Proc. 36th Int. Conf. Machine Learning (ICML), 2019, pp. 6105–6114.
[6] R. G. Allen, L. S. Pereira, D. Raes, and M. Smith, Crop Evapotranspiration: Guidelines for Computing Crop Water Requirements, FAO Irrigation and Drainage Paper No. 56. Rome, Italy: Food and Agriculture Organization of the United Nations, 1998. Available: https://www.fao.org/4/x0490e/x0490e00.htm
[7] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, “A ConvNet for the 2020s,” in Proc. 2022 IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), New Orleans, LA, USA, 2022, pp. 11966–11976, doi: 10.1109/CVPR52688.2022.01167.
[8] S. J. Czaja and C. C. Lee, “The Impact of Aging on Access to Technology,” Universal Access in the Information Society, vol. 5, no. 4, pp. 341–349, 2007. doi: 10.1007/s10209-006-0060-x. Available: https://doi.org/10.1007/s10209-006-0060-x